

PROVA DE APTIDÃO PROFISSIONAL

JOBFORALL



CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Tiago Filipe Silva

N.º: 20

PROFESSOR ORIENTADOR: Prof. Luís Fernandes

ANO LETIVO: 2025/2026



AGRADECIMENTOS

Gostaria de expressar o meu agradecimento a todas as pessoas que contribuíram, direta ou indiretamente, para a realização deste projeto de PAP.

Em primeiro lugar, agradeço aos professores da Escola Secundária Raul Proença, nas Caldas da Rainha, pelo acompanhamento, apoio e conhecimentos transmitidos ao longo do percurso académico, que foram fundamentais para o desenvolvimento deste trabalho.

Um agradecimento especial aos professores orientadores, pela orientação, disponibilidade e contributo prestado durante as várias fases do projeto.

Aos colegas, deixo também o meu agradecimento pelo apoio, partilha de ideias e espírito de entreatajuda, que contribuíram para um ambiente de trabalho positivo.

Por fim, agradeço a todos os que, de alguma forma, apoiaram e incentivaram a realização deste projeto.



ÍNDICE

Agradecimentos	I
Índice	II
Índice de figuras	V
Índice de tabelas	VII
Lista de siglas e acrónimos	VIII
1. Introdução	9
1.1. Fundamentação	9
1.2. Descrição do projeto	10
1.3. Objetivos do projeto	11
1.4. Estado do conhecimento	11
2. Planificação do projeto	13
2.1. Calendarização	13
2.1.1. Calendarização prevista	13
2.1.2. Calendarização realizada	14
2.2. Tecnologia utilizada	15
2.3. Protótipo	18
2.4. Base de Dados	21
3. Implementação do projeto	23
3.1. Preparação do ambiente de desenvolvimento	23
3.1.1. Instalação do laravel	23
3.1.2. criação do projeto jobforall	24
3.1.3. instalação do composer	25
3.1.4. instalação do node.js e npm	25
3.1.5. instalação e configuração do laragon (ambiente local de desenvolvimento)	25
3.1.6. Configuração da base de dados mysql	26
3.1.7. Instalação das dependências do projeto	27
3.2. Configuração inicial do projeto	27
3.2.1. Configuração do ficheiro .env	27
3.2.2. Ligação à base de dados mysql (via laragon)	28



3.2.3. Configuração do sistema de armazenamento (storage).....	29
3.2.4. Organização dos ficheiros públicos.....	30
3.3. Arquitetura da aplicação.....	31
3.3.1. Estrutura de pastas do laravel.....	31
3.3.2. Arquitetura MVC (Model-View-Controller).....	32
3.3.3. Separação entre Frontoffice e Backoffice.....	34
3.4. desenvolvimento da base de dados.....	34
3.4.1. Criação das migrações.....	34
3.4.2. Modelos (Models).....	35
3.4.3. Relacionamentos entre tabelas.....	35
3.5. Sistema de autenticação.....	37
3.5.1. Registo de utilizadores (Candidatos e Empresas).....	37
3.5.2. Início de Sessão.....	37
3.5.3. Gestão de perfil do utilizador.....	38
3.5.4. Proteção das áreas privadas.....	39
3.6. Sistema de permissões.....	39
3.6.1. Definição dos tipos de utilizador.....	39
3.6.2. Área de candidato.....	40
3.6.3. Área de empresa.....	41
3.6.4. Área de administrador.....	41
3.6.5. Controlo de acesso ao Backoffice.....	42
3.7. Desenvolvimento do Frontoffice (Página inicial do Utilizador e Entidade).....	42
3.7.1. Área do Utilizador (Candidato).....	43
3.7.2. Área da Entidade (Empresa).....	43
3.8. Desenvolvimento do Backoffice (Dashboard).....	44
3.8.1. Dashboard.....	44
3.8.2. Gestão de Utilizadores.....	45
3.8.3. Gestão de Empresas.....	46
3.8.4. Gestão de Ofertas de Emprego.....	47
3.8.5. Gestão de Denúncias.....	48
3.9. Definição das rotas.....	49

3.9.1. Organização do ficheiro web.php	49
3.10. Desenvolvimento dos controladores	50
3.11. Desenvolvimento das views	51
3.11.1. Blade Templates	51
3.11.2. Componentes reutilizáveis	51
3.12. Gestão de ficheiros (Imagens)	52
3.12.1. Upload de imagem dos candidatos e entidades	52
3.12.2. Armazenamento no Laragon / Storage do Laravel	53
3.13. Validação de dados	54
3.14. Funcionalidades complementares	55
3.14.1. Sistema de pesquisa e filtros	55
3.14.2. Sistemas de favoritos (Guardar Vagas)	56
3.15. Testes realizados	57
4. Problemas e soluções encontradas	58
5. Conclusão	60
Referências bibliográficas	61

ÍNDICE DE FIGURAS

Figura 1 - Logo do Visual Studio Code.....	15
Figura 2 - Logo Laragon	15
Figura 3 - Logo do Laravel	15
Figura 4 - Logo MYSQL.....	15
Figura 5 - Logo do JavaScript.....	16
Figura 6 - Logo do CSS	16
Figura 7 - Logo do HTML	16
Figura 8 - Logo do PHP.....	16
Figura 9 - Logo do BootStrap.....	17
Figura 10 - Logo do MailTrap.....	17
Figura 11 - Logo do Canva.....	17
Figura 12 - Logo do Youtube	17
Figura 13 - Logo do Copilot	18
Figura 14 - Logo do GitHub Copilot.....	18
Figura 15 - Logo do ChatGPT	18
Figura 16 - Protótipo de Login.....	18
Figura 17 - Protótipo Criar Conta.....	19
Figura 18 - Protótipo Pagina Inicial do Utilizador	19
Figura 19 - Visual final de Login	19
Figura 20 - Visual final de Criar Conta.....	20
Figura 21 - Visual final de utilizador	20
Figura 22 – Diagrama Entidade-Relação	21
Figura 23 - Tabelas de Utilizador	22
Figura 24 - Confirmação da instalação do Laravel.....	24
Figura 25 - Ativação da extensão ZIP no PHP através do Laragon.....	26
Figura 26 - Base de dados do projeto JobForAll criada no MySQL.....	27
Figura 27 - Configuração da ligação à base de dados e do serviço Mailtrap no ficheiro	28
Figura 28 - Ficheiros de migração responsáveis pela criação das tabelas da base de dados	29
Figura 29 - Ligação entre as pastas storage e public após a execução do comando php artisan storage:link.....	30
Figura 30 - Organização dos ficheiros públicos da aplicação.....	31
Figura 31 - Estrutura principal de pastas do projeto Laravel.....	32
Figura 32 - Controllers e Models	33
Figura 33 – Views	33
Figura 34 - Frontoffice	34
Figura 35 - Backoffice	34
Figura 36 - Modelos utilizados na aplicação.....	35
Figura 37 - Relacionamentos entre as tabelas da base de dados da aplicação JobForAll.....	36

Figura 38 - Formulário de registo de utilizadores	37
Figura 39 - Página de autenticação da aplicação.....	38
Figura 40 - Gestão do perfil do utilizador	38
Figura 41 - Proteção das áreas privadas através da autenticação da aplicação	39
Figura 42 - Definição dos tipos de utilizador no projeto.....	40
Figura 43 - Área reservada aos candidatos.....	40
Figura 44 - Área reservada às empresas	41
Figura 45 - Painel de administração do projeto	42
Figura 46 - Interface do utilizador no Frontoffice.....	43
Figura 47 - Interface da entidade no Frontoffice.....	44
Figura 48 - Dashboard do Backoffice com indicadores e candidaturas	45
Figura 49 - Gestão de utilizadores com filtros e controlo de contas.....	46
Figura 50 - Gestão de empresas com filtros e controlo de denúncias	47
Figura 51 - Gestão de ofertas de emprego com filtros e controlo de estados	48
Figura 52 - Sistema de gestão de denúncias com filtros e controlo de estados.....	49
Figura 53 - Organização das rotas no ficheiro web.php	50
Figura 54 - Organização dos controladores da aplicação	51
Figura 55 - Utilização de layouts e templates Blade na aplicação	52
Figura 56 - Upload das imagens para foto de perfil.....	53
Figura 57 - Armazenamento e acesso às imagens na aplicação.....	54
Figura 58 - Validação de dados	55
Figura 59 - Sistema de pesquisa e filtragem de ofertas de emprego	56
Figura 60 - Sistema de favoritos da aplicação.....	56
Figura 61 - Erro na coluna de gestão de denúncias.....	59

ÍNDICE DE TABELAS

Tabela 1 - Cronograma inicial	13
Tabela 2 - Cronograma final	14

LISTA DE SIGLAS E ACRÓNIMOS

IDE – Integrated Development Environment

PAP – Prova de Aptidão Profissional



TGP SI
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

inovação

PESSOAS
2030

PORTUGAL
2030

Cofinanciado pela
União Europeia



SELO DE
CONFORMIDADE
EQAVET

Os Fundos Europeus mais próximos de si.

1. INTRODUÇÃO

A PAP consiste na apresentação e defesa, perante um júri, de um projeto consubstanciado num produto, material ou intelectual, numa intervenção ou numa atuação, consoante a natureza dos cursos, bem como no respetivo relatório final de realização e apreciação crítica, demonstrativo de conhecimentos, aptidões, atitudes e competências profissionais adquiridas ao longo do percurso formativo do aluno, em todas as componentes de formação, com especial enfoque nas áreas de competência inscritas no Perfil dos Alunos à Saída da Escolaridade Obrigatória e no perfil profissional associado à respetiva qualificação.

O presente relatório relata as diversas etapas do desenvolvimento do projeto realizado no âmbito da PAP. Está dividido em cinco capítulos. O primeiro é esta introdução, onde são apresentadas as razões que levaram ao desenvolvimento deste projeto, bem como a descrição e os objetivos do mesmo. No segundo capítulo é a planificação do projeto, onde é apresentado a calendarização do projeto e o que foi desenvolvido ao longo do tempo de curso, como as tecnologias utilizadas a base de dados desenvolvida, os protótipos e fase final. O terceiro capítulo descreve a implementação do projeto, explicando o processo de desenvolvimento e as principais funcionalidades desenvolvidas. O quarto capítulo apresenta os problemas encontrados durante o desenvolvimento do projeto e as soluções adotadas para os resolver. Por fim, o quinto capítulo corresponde à conclusão, onde é feita uma apreciação do trabalho realizado, apresentando os resultados obtidos e uma reflexão final sobre o projeto.

1.1. FUNDAMENTAÇÃO

A ideia para o desenvolvimento do projeto JobForAll surgiu a partir da observação das dificuldades que muitas pessoas enfrentam quando procuram emprego. Atualmente, apesar de existirem várias plataformas online, o processo de procura de trabalho pode tornar-se confuso ou pouco acessível, especialmente para utilizadores que procuram uma solução simples e direta. Tendo em conta esta realidade, e após uma fase de diálogo e validação com corpo docente, foi considerado pertinente desenvolver uma plataforma que permitisse reunir, num único espaço, oportunidades de emprego e candidatos interessados.

Inicialmente, o conceito estruturado focava-se exclusivamente na divulgação de trabalhos temporários. Contudo, ao longo do planeamento, decidiu-se alargar o âmbito do projeto para o mercado de trabalho geral, permitindo abranger um maior número de setores e perfis de utilizadores. O planeamento inicial contemplava ainda a integração de um sistema de *chat* em tempo real, que visava facilitar a comunicação direta entre entidades e utilizadores, utilizadores e suporte, bem como entre entidades e suporte. No entanto, por motivos de gestão do tempo disponível e necessidade de priorizar os módulos nucleares do sistema, esta funcionalidade de conversação acabou por não ser

implementada, focando-se o projeto na estabilidade do sistema de candidaturas e na organização da informação em bases de dados.

Verifica-se que esta abordagem simplificada é de extrema relevância, pois o acesso ao emprego é um tema crucial na sociedade atual. Embora o mercado já conte com plataformas consolidadas como o LinkedIn, o Indeed ou o Net-Empregos, o JobForAll diferencia-se ao apresentar uma solução mais direta e focada na facilidade de utilização. Além disso, o desenvolvimento deste projeto proporcionou a oportunidade de aplicar na prática conhecimentos adquiridos ao longo do curso, nomeadamente na área do desenvolvimento *web*, gestão de utilizadores e administração de bases de dados.

1.2. DESCRIÇÃO DO PROJETO

O projeto JobForAll consiste no desenvolvimento de um *website* de procura e oferta de emprego. A plataforma foi criada com o objetivo de facilitar a ligação entre candidatos que procuram trabalho e empresas que pretendem divulgar oportunidades de emprego.

O sistema permite a existência de diferentes tipos de utilizadores, cada um com funcionalidades específicas. Os utilizadores candidatos podem criar uma conta na plataforma, iniciar sessão e visualizar as vagas de emprego disponíveis. Além disso, podem candidatar-se às ofertas que considerarem mais adequadas ao seu perfil, guardar vagas para consulta posterior e pesquisar ofertas por empresa, bem como proceder à edição e atualização do seu perfil pessoal.

As empresas ou entidades empregadoras também podem criar uma conta no *website*, tendo a possibilidade de inserir, editar, eliminar e gerir vagas de emprego. Desta forma, conseguem divulgar oportunidades de trabalho, encontrar potenciais candidatos interessados nessas mesmas vagas e realizar pesquisas avançadas por utilizadores registados na plataforma.

Para garantir o bom funcionamento da plataforma, existe ainda uma área de administração. Nesta secção, o administrador possui privilégios para gerir os utilizadores registados, controlar as ofertas de emprego publicadas e supervisionar o funcionamento geral do *website*. Cabe também ao administrador a moderação da plataforma, incluindo a capacidade de suspender contas e vagas em incumprimento, bem como receber e analisar denúncias submetidas pelos utilizadores e entidades.

De forma geral, o JobForAll apresenta-se como uma solução digital integrada e intuitiva, estruturada para centralizar o processo de recrutamento de ponta a ponta. Através da clara distinção de permissões entre candidatos, empresas e administradores, o sistema assegura um ambiente seguro, organizado e eficiente para a interação entre a oferta e a procura no mercado de trabalho.

1.3. OBJETIVOS DO PROJETO

O principal objetivo do projeto JobForAll é o desenvolvimento de uma plataforma *online* que facilite a ligação entre indivíduos que procuram emprego e empresas que pretendem recrutar novos colaboradores. Através deste *website*, pretende-se tornar o processo de procura e divulgação de oportunidades de trabalho mais simples, organizado e acessível.

Uma das principais mais-valias do projeto reside na possibilidade de os candidatos encontrarem e candidatarem-se a ofertas de emprego de forma rápida e prática, sendo apenas necessário o registo de uma conta na plataforma para obter acesso às vagas disponíveis. Desta forma, torna-se possível a consulta de diferentes oportunidades de trabalho num único local.

Para as empresas, a plataforma oferece a capacidade de divulgar ofertas de emprego e encontrar potenciais candidatos, permitindo a gestão das vagas publicadas de forma simplificada, através da inserção, edição ou remoção de anúncios.

Adicionalmente, o projeto apresenta como mais-valia a integração de uma área de administração. Esta componente permite o controlo e a gestão do funcionamento geral do *website*, garantindo uma maior organização da informação estruturada e dos utilizadores registados no sistema.

1.4. ESTADO DO CONHECIMENTO

Antes de iniciar o desenvolvimento do projeto JobForAll, foi necessário consolidar e aprofundar diversos conhecimentos na área do desenvolvimento web.

Ao longo do curso, já tinha adquirido bases em linguagens como HTML, CSS e JavaScript, que foram essenciais para a construção da estrutura, estilo e interatividade da aplicação. Da mesma forma, os conhecimentos de MySQL já tinham sido abordados anteriormente, permitindo compreender a criação e gestão de bases de dados.

No entanto, o desenvolvimento deste projeto exigiu um maior aprofundamento de conhecimentos, especialmente no que diz respeito ao *framework* Laravel. Apesar de ter tido apenas uma introdução ao Laravel em contexto de sala de aula, foi necessário explorar de forma mais autónoma as suas funcionalidades, incluindo a organização do projeto, criação de rotas, controllers, models e implementação de sistemas de autenticação.

Adicionalmente, foi também utilizado o *framework* Bootstrap, que facilitou a construção de interfaces modernas e responsivas, melhorando a experiência do utilizador e tornando o desenvolvimento mais eficiente.

Para complementar estes conhecimentos, foi ainda necessário recorrer à documentação online, tutoriais e vídeos explicativos, que permitiram ultrapassar dificuldades e aprofundar a compreensão das tecnologias utilizadas.

Desta forma, o desenvolvimento do projeto contribuiu significativamente para o reforço e aquisição de novos conhecimentos, sobretudo na utilização prática do Laravel e na integração de diferentes tecnologias no desenvolvimento de uma aplicação web completa.



TGP SI
CURSO PROFISSIONAL



Associação



Os Fundos Europeus mais próximos de si.



2. PLANIFICAÇÃO DO PROJETO

2.1. CALENDARIZAÇÃO

A primeira etapa consistiu na elaboração de um cronograma com todas as fases do projeto e respetivas durações tendo em conta a data estipulada para entrega do mesmo.

2.1.1. CALENDARIZAÇÃO PREVISTA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema									
Esboço dos templates									
Pesquisar os templates									
Ajustar os templates									
Passar para a Framework Laravel									
Construção da base de dados									
Lógica de autenticação									
Lógica das páginas do utilizador									
Lógica das páginas da entidade									
Lógica das páginas do administrador									
Correção de erros									

Tabela 1 - Cronograma inicial

2.1.2. CALENDARIZAÇÃO REALIZADA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema									
Esboço dos templates									
Pesquisar os templates									
Ajustar os templates									
Passar para a Framework Laravel									
Construção da base de dados									
Lógica de autenticação									
Lógica das páginas do utilizador									
Lógica das páginas da entidade									
Lógica das páginas da administração									
Correção de erros									

Tabela 2 - Cronograma final

2.2. TECNOLOGIA UTILIZADA

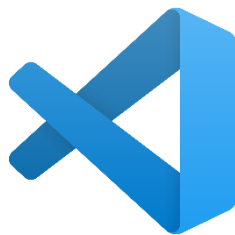


Figura 1 - Logo do Visual Studio Code

O Visual Studio Code, foi o software utilizado para editar código.



Figura 2 - Logo Laragon

O Laragon foi utilizado para fazer a ligação da base de dados a plataforma.

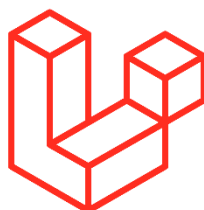


Figura 3 - Logo do Laravel

O Laravel foi o framework utilizada para deixar o site muito mais completo, seguro e organizado.



Figura 4 - Logo MYSQL

MySQL Workbench foi o programa utilizado para desenvolvimento da base de dados.

JavaScript



Figura 5 - Logo do JavaScript

O JavaScript foi para fazer algumas funcionalidades mais simples.

CSS



Figura 6 - Logo do CSS

O CSS foi utilizado para deixar o site muito mais bonito visualmente.

HTML



Figura 7 - Logo do HTML

O HTML foi utilizado para fazer os templates, para o projeto final.



Figura 8 - Logo do PHP

O PHP foi a linguagem de programação usada para fazer a maior parte da lógica da plataforma.



Figura 9 - Logo do Bootstrap

O Bootstrap foi utilizado para deixar a plataforma responsiva com os diversos dispositivos.



Figura 10 - Logo do MailTrap

O MailTrap foi utilizado para fazer simulações de emails.



Figura 11 - Logo do Canva

O Canva foi utilizado para fazer o esboço dos protótipos.



Figura 12 - Logo do Youtube

O Youtube foi utilizado para pesquisa de templates e código para a framework Laravel.



Figura 13 - Logo do Copilot

O Copilot foi utilizado para criação do logo do projeto.



Figura 14 - Logo do GitHub Copilot

O GitHub Copilot foi utilizado como suporte em partes de código e alguns erros.



Figura 15 - Logo do ChatGPT

O ChatGPT foi utilizado como suporte para preencher a tabela de skills da base de dados.

2.3. PROTÓTIPO



Figura 16 - Protótipo de Login

Protótipo de tela para criação de conta. O formulário contém campos para NOME, PASSWORD e CONFIRME A PASSWORD. Abaixo dos campos, há um link: "Caso já tenha uma conta, pode fazer login [aqui](#).". No final, há um botão "CRIAR".

Figura 17 - Protótipo Criar Conta

Protótipo de tela de perfil de usuário. À esquerda, um cartão de perfil com o nome "Humberto", email "Humberto@gmail.com", telefone "123456789", escolaridade "12º Ano", experiências "Caixas de supermercado" e cursos "Informática". À direita, uma barra de navegação com ícones de busca, menu e perfil. Abaixo, três cartões de oferta: "Loja Informática", "LIDL" e "Escola de Condução", cada um com uma imagem e texto descritivo.

Figura 18 - Protótipo Pagina Inicial do Utilizador

Antes da fase de codificação, foram realizados pequenos "rabiscos" preliminares, visíveis nas imagens acima. Contudo, assim que o código foi desenvolvido e integrado na *framework*, o design final divergiu completamente do protótipo inicial, conforme demonstrado nas imagens abaixo.

Visual final de Login. A tela é escura com um formulário centralizado. No topo, há um ícone de usuário e o texto "JOB FOR ALL". Abaixo, o título "Entrar". O formulário contém campos para "Email" e "Senha", um checkbox "Lembrar-me" e um link "Esqueci minha senha". No final, há um botão "Log in" e um link "Não tem conta? Criar conta".

Figura 19 - Visual final de Login

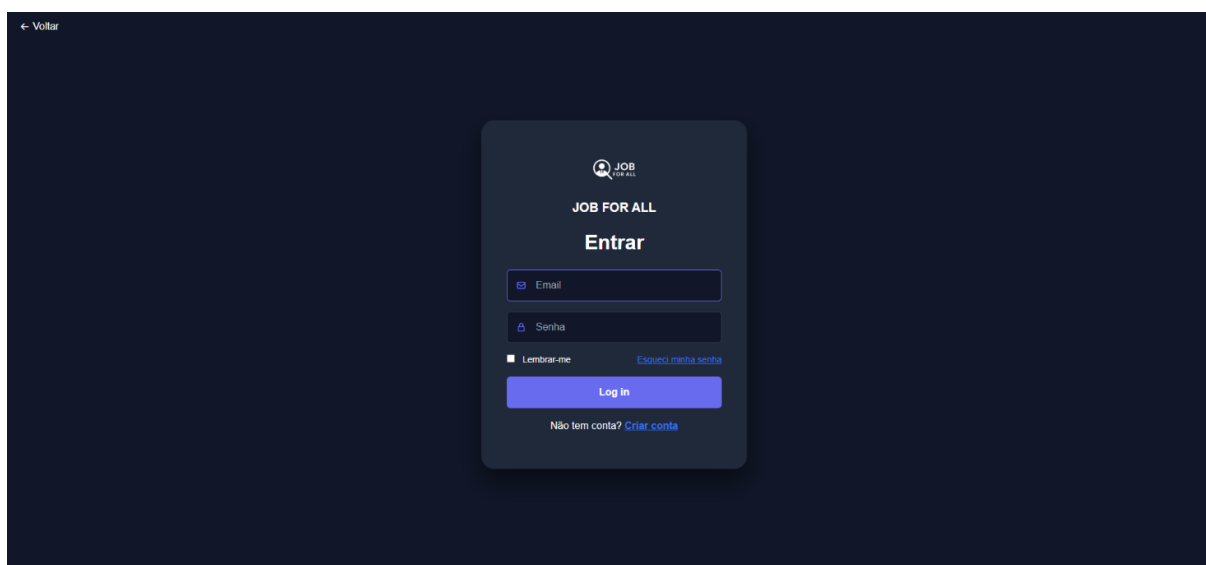


Figura 20 - Visual final de Criar Conta

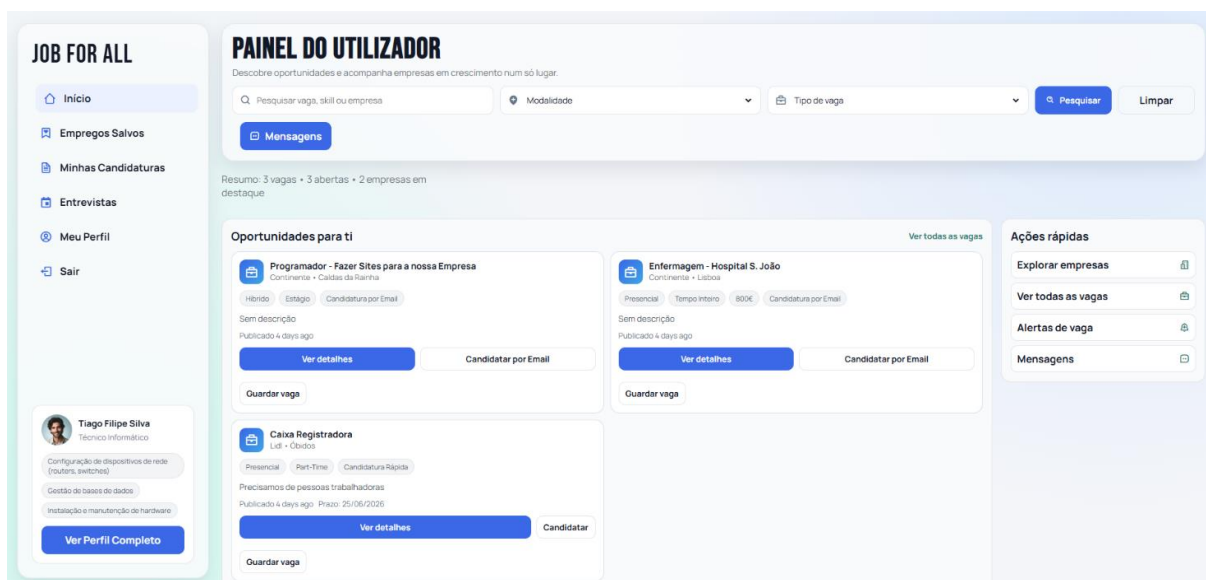


Figura 21 - Visual final de utilizador

Como se pode ver na fase final ficou completamente diferente e muito mais agradável visualmente comparado ao protótipo inicial.

2.4. BASE DE DADOS

Durante o desenvolvimento do projeto, foi necessária a construção de uma base de dados. Embora algumas tabelas pudessem ter sido omitidas, optou-se pela separação estrutural dos dados, como ilustra o exemplo da gestão de utilizadores apresentado na figura abaixo. A base de dados é composta por um total de 24 tabelas, das quais 5 não possuem qualquer relação, uma vez que são nativas da própria *framework* utilizada. As restantes tabelas destinam-se à gestão de utilizadores e respetivos perfis, vagas de trabalho, entidades e denúncias.

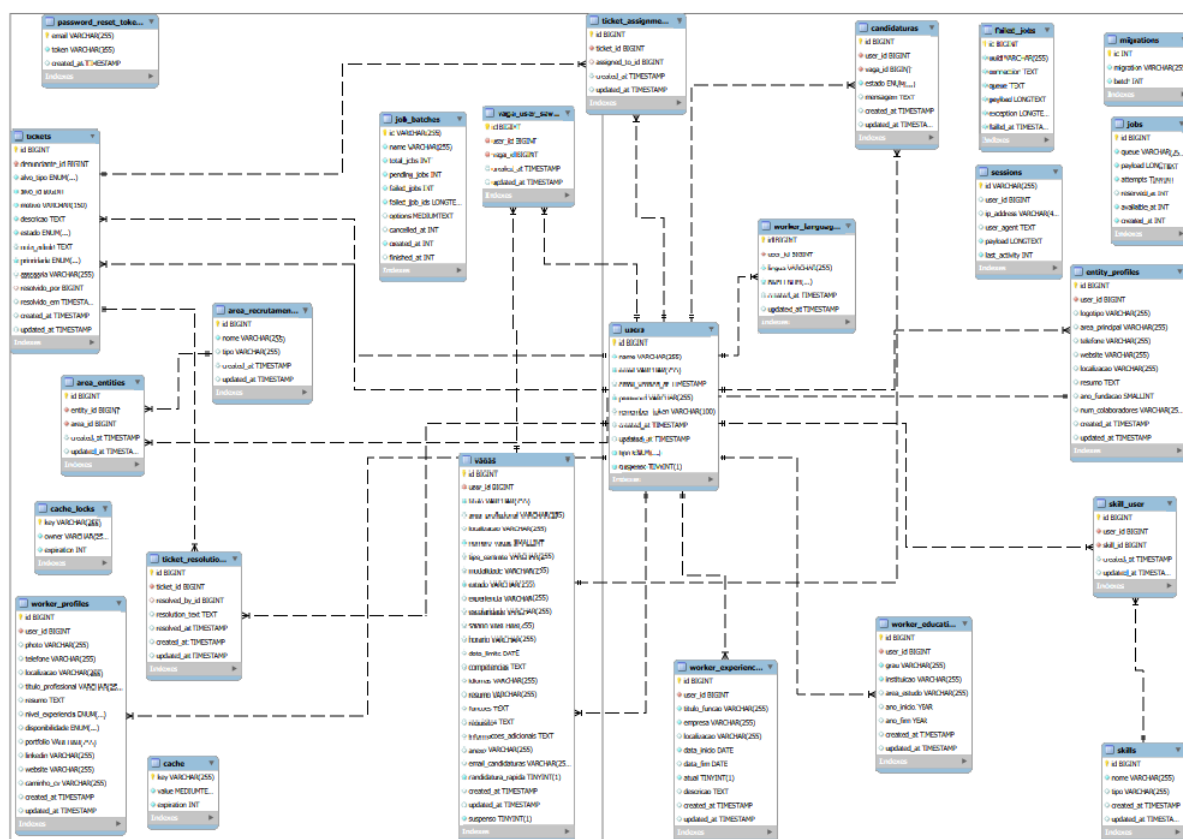


Figura 22 – Diagrama Entidade-Relação

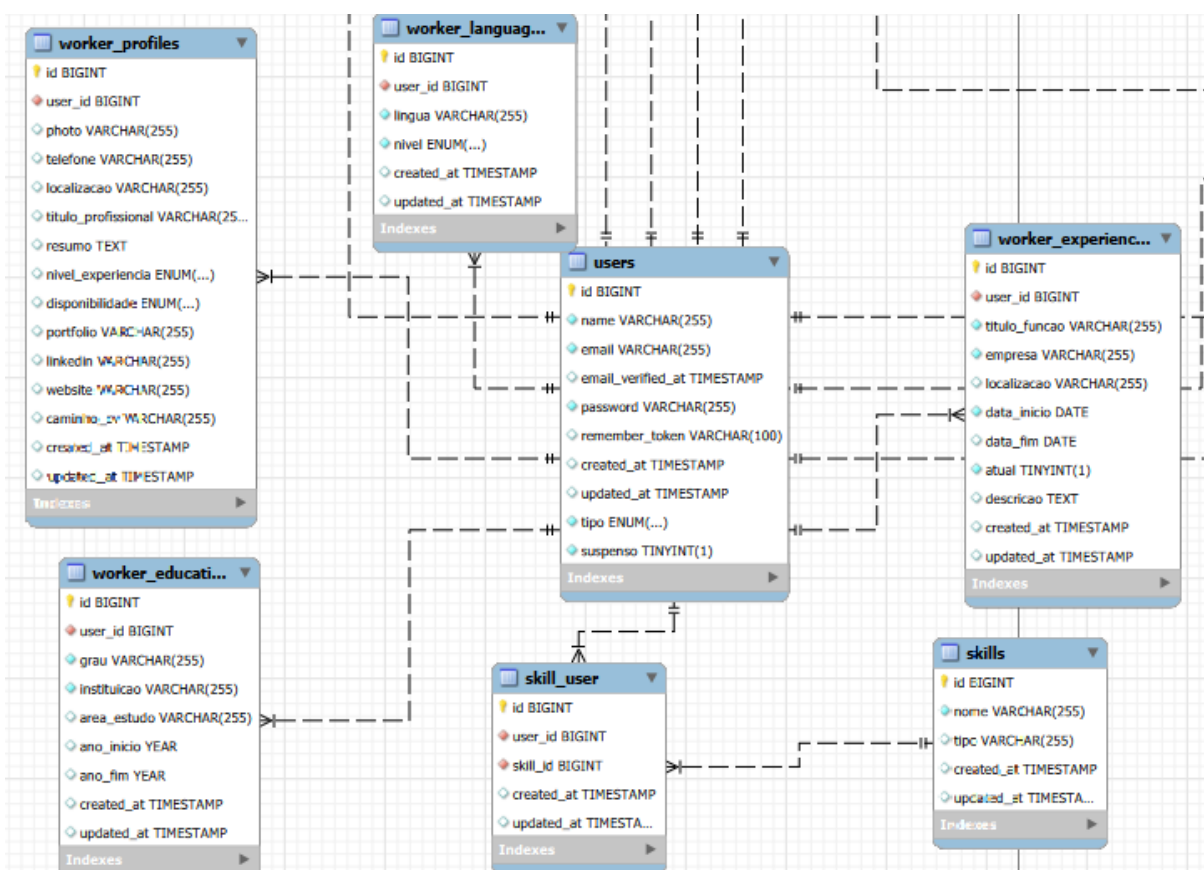


Figura 23 - Tabelas de Utilizador

Como se pode observar, a entidade do utilizador foi estruturada e dividida em sete tabelas distintas, sendo a tabela *users* a tabela principal. Esta é responsável pelo armazenamento dos dados de autenticação (correio eletrónico e palavra-passe) e pelo controlo do estado da conta (por exemplo, se se encontra suspensa ou ativa). A tabela *worker_language* é utilizada para registar e associar os idiomas falados pelo utilizador, bem como o respetivo nível de proficiência (como, por exemplo, "Português – Nativo"), permitindo a respetiva edição ou atualização.

Na tabela *worker_profiles*, são centralizadas as informações de carácter pessoal e biográfico do utilizador, tais como o nível de experiência, uma breve descrição curricular e a área de residência. Complementarmente, a tabela *worker_education* destina-se ao registo das qualificações académicas, nível de escolaridade e cursos concluídos, enquanto a tabela *worker_experience* permite mapear o histórico profissional e a experiência adquirida no mercado de trabalho.

Por fim, a estrutura é completada por duas tabelas dedicadas às competências técnicas. A tabela *skills* encontra-se previamente populada com diversas competências categorizadas por áreas profissionais; para delimitar o âmbito do projeto, foram integradas três áreas distintas: Informática, Saúde e Gestão. Dado que um utilizador pode possuir múltiplas competências, foi criada a tabela relacional intermédia *skills_user*, que estabelece de forma eficiente a associação de muitos-para-muitos entre os utilizadores e as suas respetivas qualificações.

3. IMPLEMENTAÇÃO DO PROJETO

Neste capítulo é descrito o processo de implementação do projeto JobForAll, desde a preparação do ambiente de desenvolvimento até à configuração das ferramentas e tecnologias necessárias para o seu funcionamento. São apresentados os procedimentos realizados para garantir uma base de desenvolvimento estável, organizada e adequada à criação da aplicação. Nos subcapítulos seguintes são detalhadas as diferentes etapas da implementação.

3.1. PREPARAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

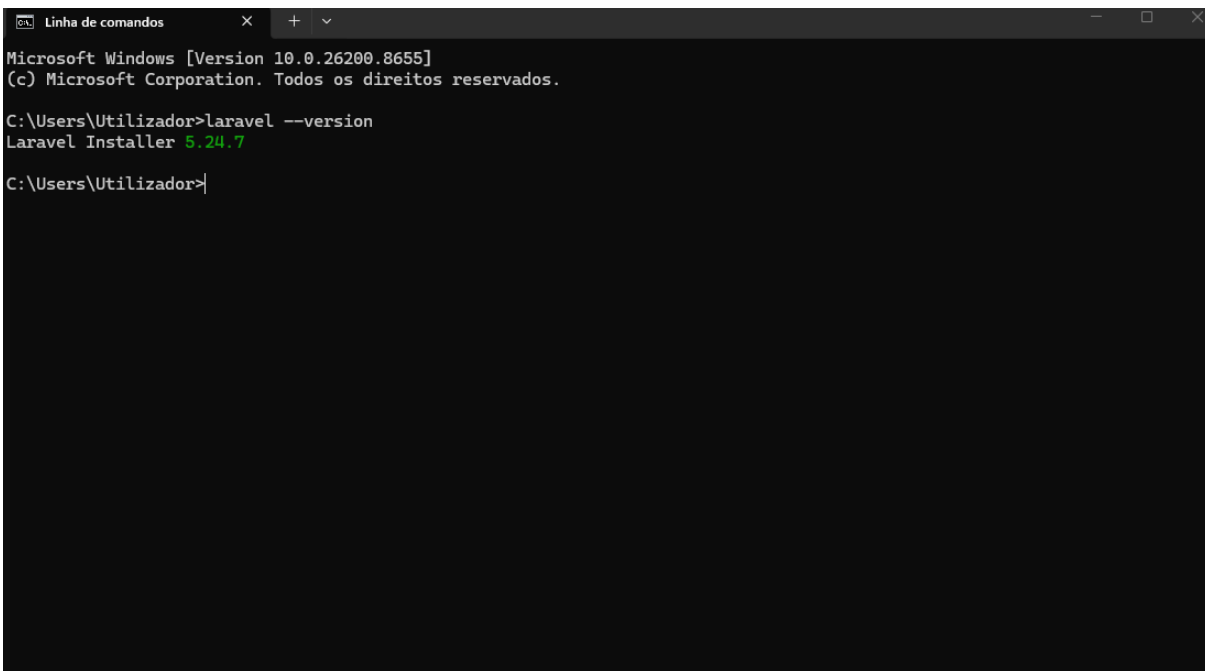
Antes do desenvolvimento da aplicação, foi necessário preparar um ambiente de trabalho que permitisse desenvolver, testar e executar o projeto de forma eficiente. Para esse efeito, procedeu-se à instalação e configuração das principais ferramentas utilizadas ao longo do desenvolvimento, nomeadamente o Laravel, o Composer, o Node.js, o NPM, o Visual Studio Code, o Laragon e o sistema de gestão de bases de dados MySQL. Além disso, foram instaladas todas as dependências necessárias ao correto funcionamento do projeto.

3.1.1. INSTALAÇÃO DO LARAVEL

O Laravel foi o *framework* escolhido para o desenvolvimento da aplicação JobForAll, devido à sua robustez, organização e conjunto de funcionalidades que facilitam o desenvolvimento de aplicações web. Antes da criação do projeto, foi necessário garantir que todos os requisitos do Laravel se encontravam instalados, nomeadamente o PHP, o Composer e as respetivas extensões.

A instalação foi realizada através do Composer, utilizando o comando apropriado no terminal. Após a conclusão do processo, foi possível confirmar que o Laravel se encontrava corretamente instalado e pronto para ser utilizado na criação do projeto.

A correta instalação do Laravel constituiu uma etapa essencial, uma vez que este *framework* fornece a estrutura base da aplicação, permitindo uma organização eficiente do código e facilitando a implementação das funcionalidades previstas.



```
Linha de comandos
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Utilizador>laravel --version
Laravel Installer 5.24.7

C:\Users\Utilizador>
```

Figura 24 - Confirmação da instalação do Laravel.

3.1.2. CRIAÇÃO DO PROJETO JOBFORALL

Após a instalação do Laravel, procedeu-se à criação do projeto, ao qual foi atribuído o nome JobForAll. Esta etapa permitiu gerar automaticamente toda a estrutura inicial da aplicação, incluindo as pastas, ficheiros e configurações necessárias ao desenvolvimento.

A criação foi efetuada através do terminal utilizando o comando disponibilizado pelo Laravel. Depois de concluído o processo, verificou-se que a estrutura do projeto tinha sido criada corretamente e que a aplicação podia ser executada localmente.

Esta estrutura inicial serviu de base para o desenvolvimento das diferentes funcionalidades implementadas ao longo do projeto.

3.1.3. INSTALAÇÃO DO COMPOSER

O Composer é o gestor de dependências utilizado pelo PHP e desempenha um papel fundamental no desenvolvimento de aplicações Laravel. A sua principal função consiste em instalar, atualizar e gerir automaticamente as bibliotecas utilizadas pelo projeto.

Após a instalação do Composer, foi possível instalar todas as dependências definidas no ficheiro `composer.json`, garantindo que o projeto possuía todos os componentes necessários ao seu funcionamento.

A utilização desta ferramenta simplifica significativamente a gestão das bibliotecas externas e assegura a compatibilidade entre as diferentes versões dos pacotes utilizados.

3.1.4. INSTALAÇÃO DO NODE.JS E NPM

Para além das dependências PHP, o projeto necessita também de ferramentas destinadas à gestão dos recursos gráficos, como folhas de estilo CSS e ficheiros JavaScript. Para esse efeito, foi instalado o Node.js, que inclui automaticamente o NPM (Node Package Manager).

Posteriormente, através do NPM, foram instaladas as dependências necessárias ao frontend da aplicação, permitindo compilar e gerir os recursos utilizados durante o desenvolvimento.

Esta configuração possibilitou a utilização de tecnologias modernas para o desenvolvimento da interface gráfica da aplicação.

3.1.5. INSTALAÇÃO E CONFIGURAÇÃO DO LARAGON (AMBIENTE LOCAL DE DESENVOLVIMENTO)

Para a execução da aplicação em ambiente local foi utilizado o Laragon, uma vez que esta ferramenta disponibiliza um ambiente de desenvolvimento completo, integrando serviços como o Apache, PHP e MySQL. Após a instalação do Laragon, procedeu-se à configuração do ambiente de desenvolvimento de forma a garantir a compatibilidade com o Laravel. Durante este processo, foi necessário ativar a extensão ZIP do PHP, através da remoção do comentário da diretiva correspondente no ficheiro de configuração `php.ini`. Esta alteração permitiu disponibilizar as funcionalidades de compressão exigidas por algumas dependências do Laravel e do Composer.

Concluída esta configuração, os serviços do Laragon foram reiniciados, permitindo que todas as alterações fossem aplicadas corretamente. Desta forma, ficou preparado um ambiente de desenvolvimento estável para a implementação e execução da aplicação.

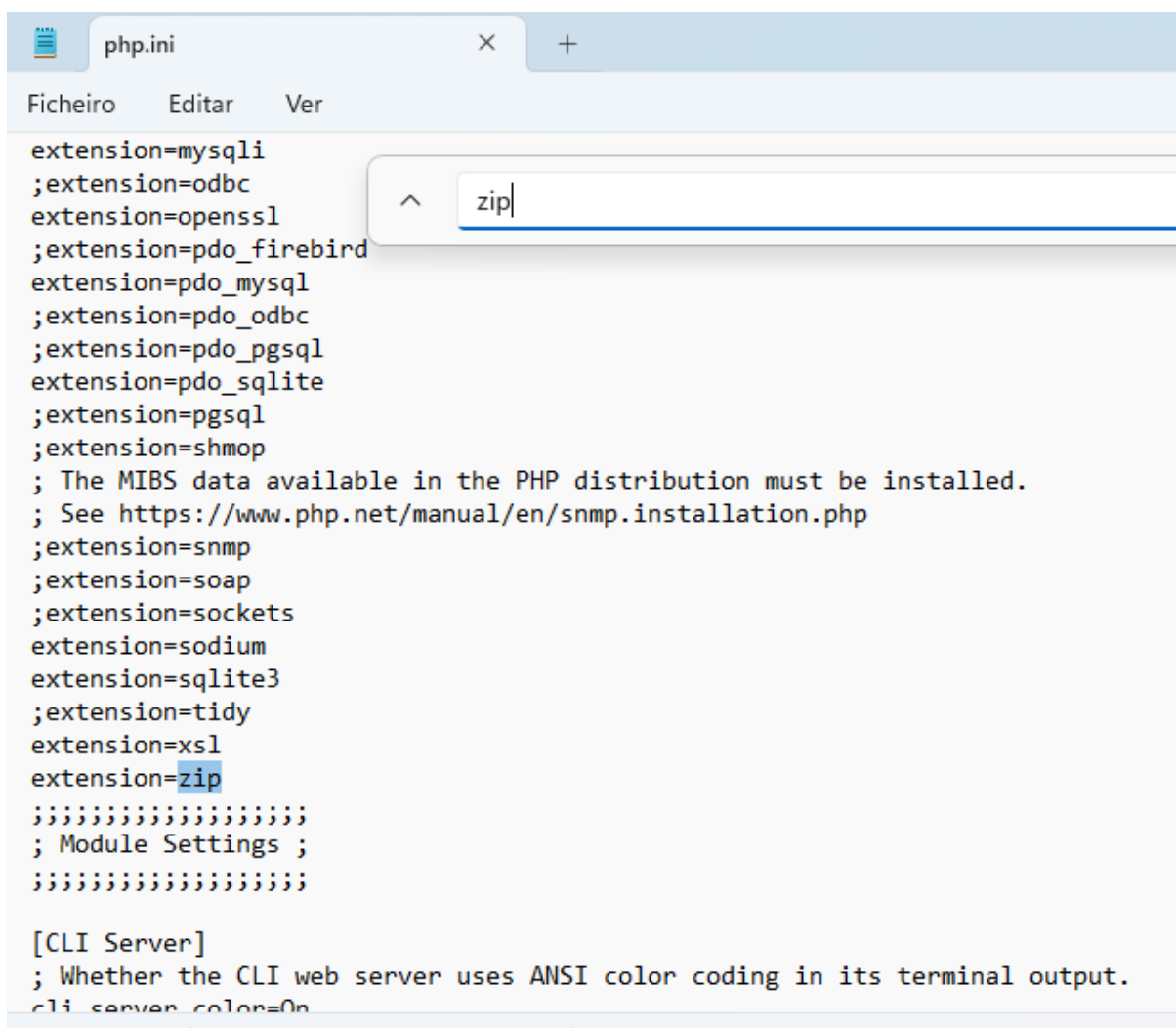


Figura 25 - Ativação da extensão ZIP no PHP através do Laragon.

3.1.6. CONFIGURAÇÃO DA BASE DE DADOS MYSQL

Após a preparação do ambiente de desenvolvimento, foi criada a base de dados MySQL destinada ao projeto JobForAll. Esta base de dados passou a ser utilizada para armazenar toda a informação da aplicação, incluindo utilizadores, ofertas de emprego, candidaturas e restantes dados necessários ao seu funcionamento.

Depois da criação da base de dados, a ligação entre o Laravel e o MySQL foi configurada, permitindo que a aplicação comunicasse corretamente com o sistema de gestão de bases de dados. Posteriormente, foram executadas as migrações do Laravel, responsáveis pela criação automática das tabelas necessárias ao funcionamento da aplicação.

The screenshot shows a MySQL database interface. On the left, the 'jobforall' database schema is visible, listing various tables like 'area_entities', 'area_recrutamento', 'cache', 'cache_locks', 'candidatures', 'entity_profiles', 'failed_jobs', 'job_batches', 'jobs', 'migrations', 'password_reset_tokens', 'sessions', 'skill_user', 'skills', 'ticket_assignments', 'ticket_resolutions', 'tickets', 'users', 'vaga_user_saved', 'vagas', 'worker_education', 'worker_experiences', 'worker_languages', and 'worker_profiles'. The main area displays a query: `select * from users`. The result is a table with 10 columns: `id`, `name`, `email`, `email_verified_at`, `password`, `remember_token`, `created_at`, `updated_at`, `tipo`, and `suspenso`. The table contains 6 rows of data.

id	name	email	email_verified_at	password	remember_token	created_at	updated_at	tipo	suspenso
1	Tiago Filipe Silva	asid@gmail.com	2026-05-30 18:47:44	\$2y\$12\$BqQdQ2Htz4LJ0NDX0Bpu4Hm7q4Uj3oZfym1CMLArpQ7i	2026-05-30 18:47:44	2026-05-30 18:47:44	2026-05-30 18:47:44	user	0
2	Lid	lid@lid	2026-05-30 18:51:44	\$2y\$12\$BqQdQ2Htz4LJ0NDX0Bpu4Hm7q4Uj3oZfym1CMLArpQ7i	2026-05-30 18:51:44	2026-05-30 18:51:44	2026-05-30 18:51:44	entidade	0
3	Admin	admin@admin	2026-05-30 18:53:28	\$2y\$12\$BqQdQ2Htz4LJ0NDX0Bpu4Hm7q4Uj3oZfym1CMLArpQ7i	2026-05-30 18:53:28	2026-05-30 18:53:28	2026-05-30 18:53:28	admin	0
5	Ana	ana@ana	2026-05-30 19:12:37	\$2y\$12\$BqQdQ2Htz4LJ0NDX0Bpu4Hm7q4Uj3oZfym1CMLArpQ7i	2026-05-30 19:12:37	2026-05-30 19:12:37	2026-05-30 19:12:37	user	0
6	Contnente	contnente@contnente	2026-06-19 21:52:42	\$2y\$12\$BqQdQ2Htz4LJ0NDX0Bpu4Hm7q4Uj3oZfym1CMLArpQ7i	2026-06-19 21:52:42	2026-06-19 21:52:42	2026-06-19 21:52:42	entidade	0

Figura 26 - Base de dados do projeto JobForAll criada no MySQL.

3.1.7. INSTALAÇÃO DAS DEPENDÊNCIAS DO PROJETO

Após a criação do projeto e da configuração das ferramentas necessárias, procedeu-se à instalação de todas as dependências utilizadas pela aplicação. Foram executados os comandos `composer install` e `npm install`, permitindo instalar automaticamente todas as bibliotecas necessárias ao funcionamento do *backend* e do *frontend*.

Posteriormente, foi executado o comando responsável pela compilação dos recursos da aplicação, garantindo que os ficheiros CSS e JavaScript ficaram preparados para utilização durante o desenvolvimento. A conclusão desta etapa permitiu iniciar o desenvolvimento das funcionalidades previstas para o projeto, assegurando que todas as dependências estavam corretamente instaladas e configuradas.

3.2. CONFIGURAÇÃO INICAL DO PROJETO

Após a preparação do ambiente de desenvolvimento, foi necessário proceder à configuração inicial da aplicação Laravel. Nesta fase foram definidos os principais parâmetros de funcionamento do projeto, garantindo a correta comunicação com a base de dados, a gestão dos ficheiros da aplicação e a organização dos recursos utilizados durante o desenvolvimento.

3.2.1. CONFIGURAÇÃO DO FICHEIRO .ENV

O ficheiro `.env` foi utilizado para configurar os principais parâmetros necessários ao funcionamento da aplicação durante o desenvolvimento. Neste projeto, foram alteradas as configurações de ligação à base de dados MySQL e do serviço de envio de correio eletrónico através do Mailtrap.

A configuração da base de dados permitiu estabelecer a comunicação entre o Laravel e o MySQL, enquanto a integração com o Mailtrap possibilitou o teste do envio de mensagens de correio eletrónico sem recorrer a um servidor de produção. Esta abordagem permitiu validar funcionalidades como o envio de notificações e mensagens de forma segura durante o desenvolvimento da aplicação.

```

23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=jobforall
27 DB_USERNAME=root
28 DB_PASSWORD=root
29
30 SESSION_DRIVER=database
31 SESSION_LIFETIME=120
32 SESSION_ENCRYPT=false
33 SESSION_PATH=/
34 SESSION_DOMAIN=null
35
36 BROADCAST_CONNECTION=log
37 FILESYSTEM_DISK=local
38 QUEUE_CONNECTION=database
39
40 CACHE_STORE=database
41 # CACHE_PREFIX=
42
43 MEMCACHED_HOST=127.0.0.1
44
45 REDIS_CLIENT=phpredis
46 REDIS_HOST=127.0.0.1
47 REDIS_PASSWORD=null
48 REDIS_PORT=6379
49
50 MAIL_MAILER=smtp
51 MAIL_SCHEME=null
52 MAIL_HOST=sandbox.smtp.mailtrap.io
53 MAIL_PORT=2525
54 MAIL_USERNAME=71d49a7100332e
55 MAIL_PASSWORD=bbbfeda197c294
56 MAIL_FROM_ADDRESS="JobForALL@company.com"
57 MAIL_FROM_NAME="${APP_NAME}"

```

Figura 27 - Configuração da ligação à base de dados e do serviço Mailtrap no ficheiro

3.2.2. LIGAÇÃO À BASE DE DADOS MYSQL (VIA LARAGON)

Depois de criada a base de dados, foi estabelecida a ligação entre o Laravel e o servidor MySQL disponibilizado pelo Laragon. Esta configuração permitiu que a aplicação armazenasse e consultasse os dados de forma automática. Após a configuração da ligação, foi executado o comando de migração (php artisan migrate), responsável pela criação das tabelas definidas no projeto.

A conclusão desta etapa garantiu que todas as operações relacionadas com a base de dados pudessem ser realizadas corretamente durante o desenvolvimento da aplicação.

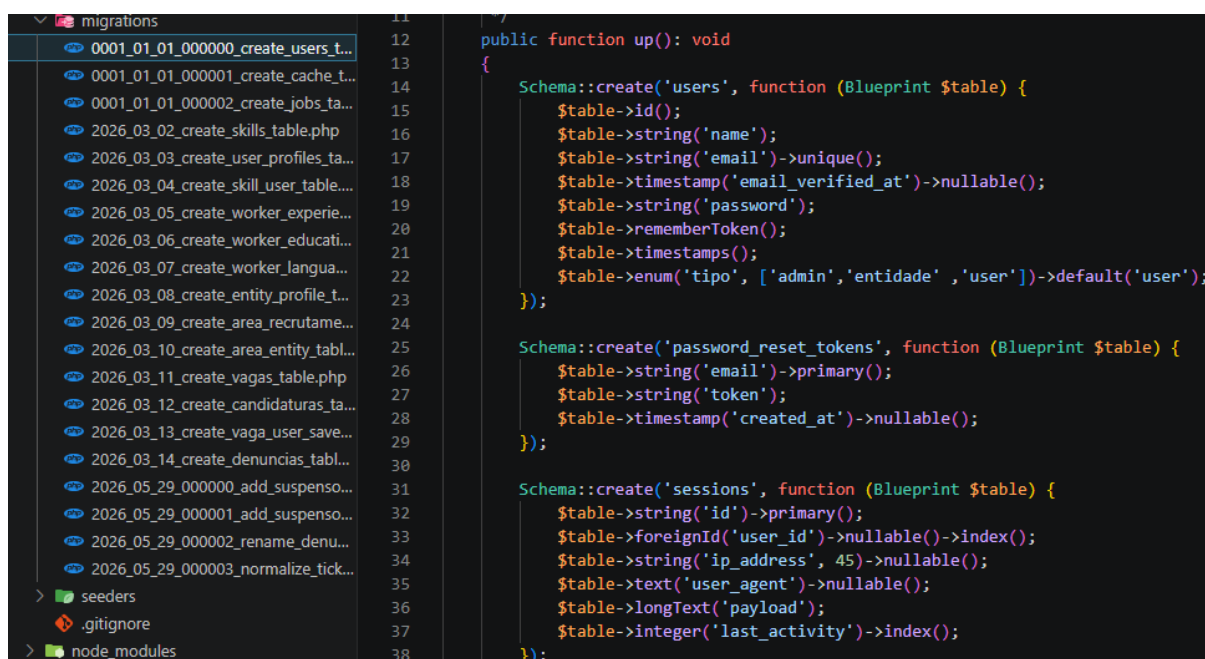


Figura 28 - Ficheiros de migração responsáveis pela criação das tabelas da base de dados

3.2.3. CONFIGURAÇÃO DO SISTEMA DE ARMAZENAMENTO (STORAGE)

Durante o desenvolvimento da aplicação foi necessário configurar o sistema de armazenamento de ficheiros do Laravel, permitindo guardar imagens e outros documentos enviados pelos utilizadores. Para esse efeito, foi criado o atalho simbólico entre a pasta storage e a pasta public, através do comando `php artisan storage:link`. Esta configuração tornou os ficheiros armazenados acessíveis através do navegador, possibilitando a sua apresentação na interface da aplicação.

Esta funcionalidade foi utilizada para armazenar conteúdos associados aos utilizadores e restantes elementos da plataforma.

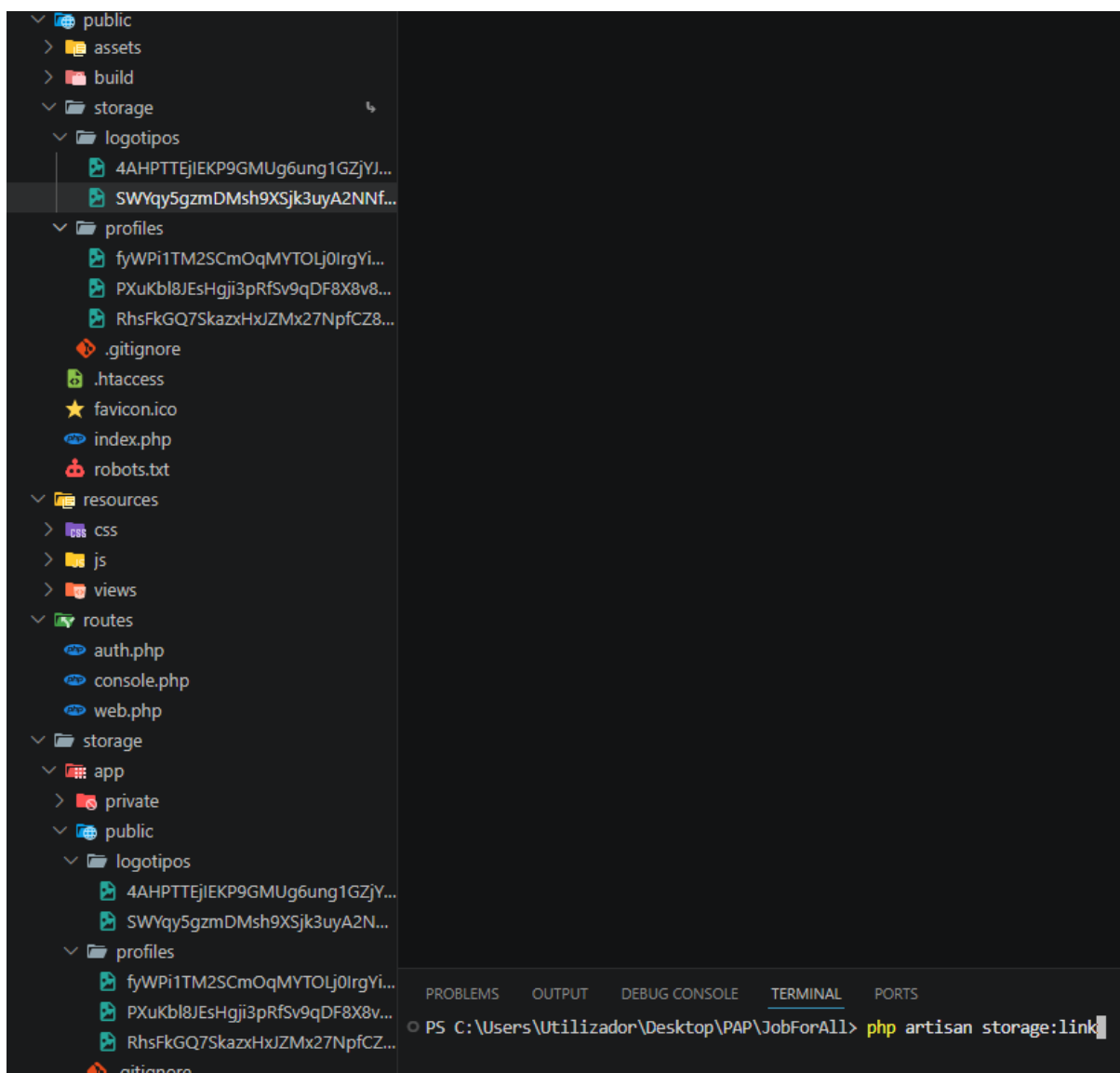


Figura 29 - Ligação entre as pastas storage e public após a execução do comando `php artisan storage:link`

3.2.4. ORGANIZAÇÃO DOS FICHEIROS PÚBLICOS

Após a configuração inicial da aplicação, procedeu-se à organização dos ficheiros públicos utilizados pelo projeto.

A pasta `public` foi utilizada para armazenar os recursos acessíveis diretamente pelos utilizadores, tais como folhas de estilo (CSS), ficheiros JavaScript, imagens e outros conteúdos estáticos. Os ficheiros enviados pelos utilizadores ficaram armazenados na pasta `storage`, sendo disponibilizados através da ligação criada anteriormente. Esta organização permitiu manter uma estrutura de projeto organizada, facilitando a manutenção do código e a gestão dos diferentes recursos utilizados pela aplicação.

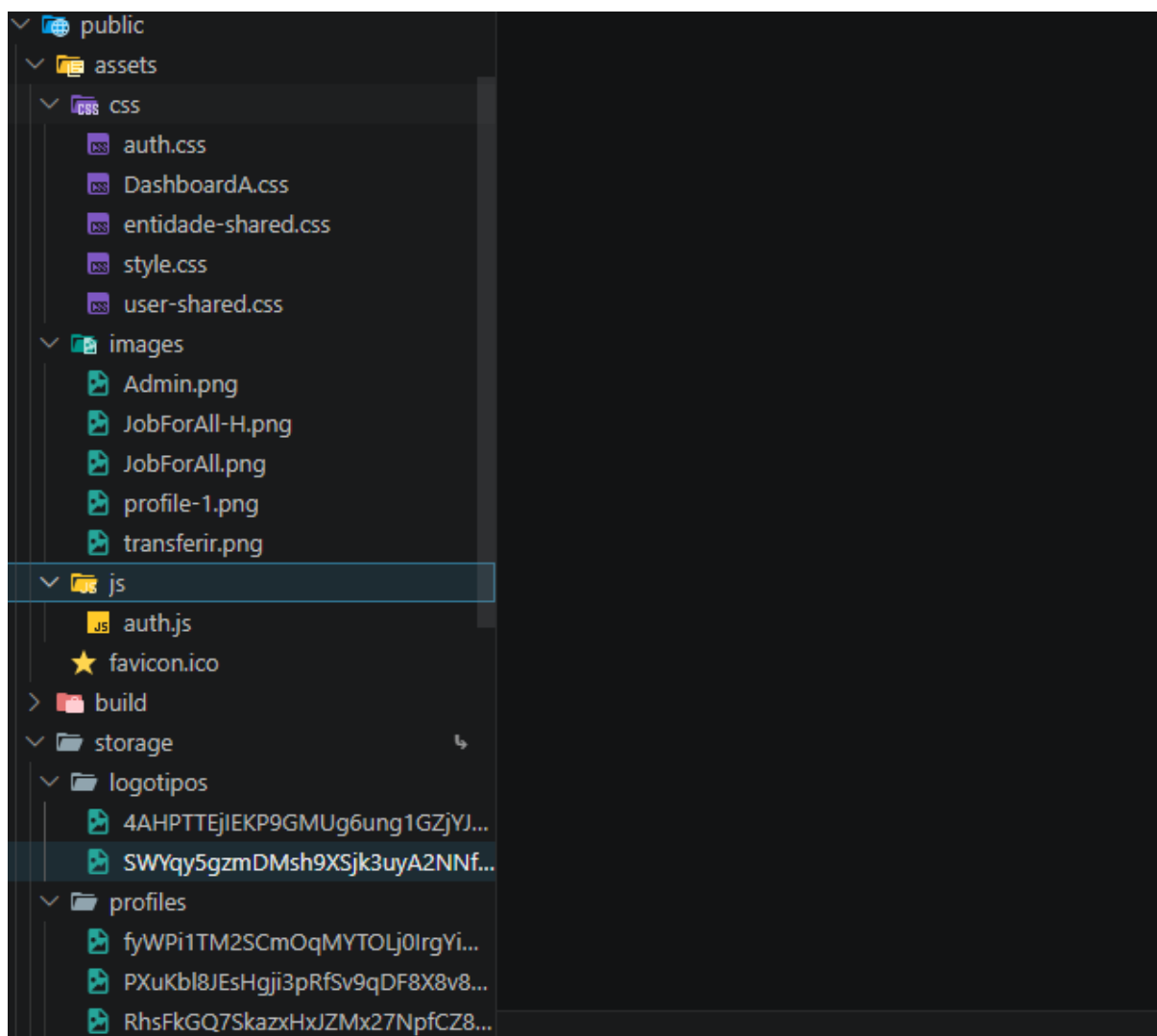


Figura 30 - Organização dos ficheiros públicos da aplicação

3.3. ARQUITETURA DA APLICAÇÃO

Durante o desenvolvimento do projeto JobForAll, foi adotada a arquitetura disponibilizada pelo framework Laravel. Esta estrutura permitiu organizar o código de forma modular, facilitando o desenvolvimento, a manutenção e a implementação de novas funcionalidades. Nos pontos seguintes são apresentadas as principais características da organização da aplicação.

3.3.1. ESTRUTURA DE PASTAS DO LARAVEL

O Laravel disponibiliza uma estrutura de pastas organizada, onde cada diretório possui uma função específica dentro da aplicação. Durante o desenvolvimento do projeto, as pastas mais

utilizadas foram `app`, responsável pela lógica da aplicação; `resources`, onde se encontram as vistas (Views); `routes`, destinada à definição das rotas; `database`, que contém as migrações; `public`, utilizada para disponibilizar os ficheiros públicos da aplicação; e `storage`, destinada ao armazenamento de ficheiros enviados pelos utilizadores.

Esta organização facilitou a localização dos diferentes componentes do projeto, contribuindo para uma maior organização e facilidade de manutenção do código.

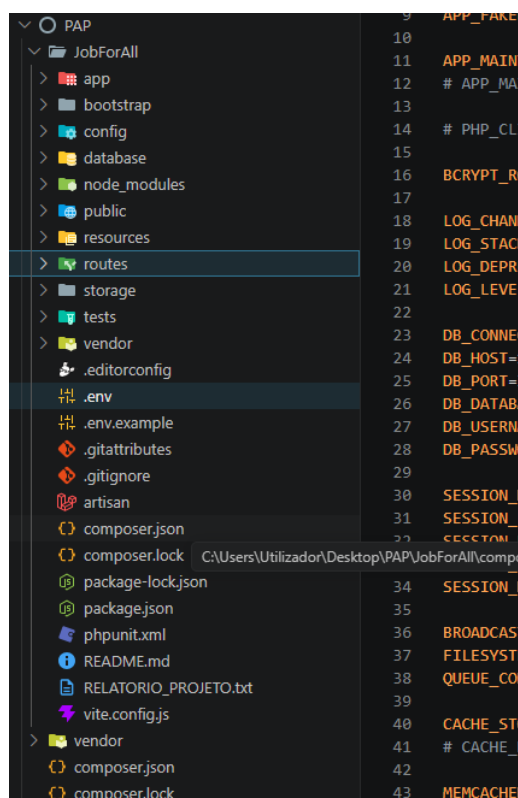


Figura 31 - Estrutura principal de pastas do projeto Laravel

3.3.2. ARQUITETURA MVC (MODEL-VIEW-CONTROLLER)

O projeto foi desenvolvido seguindo a arquitetura Model-View-Controller (MVC), padrão utilizado pelo Laravel para separar as diferentes responsabilidades da aplicação.

Os Models foram responsáveis pela comunicação com a base de dados e pela representação das entidades do sistema. As Views foram utilizadas para apresentar a interface gráfica aos utilizadores, recorrendo ao motor de templates Blade. Os Controllers receberam os pedidos efetuados pelos utilizadores, processaram a lógica da aplicação e estabeleceram a comunicação entre os Models e as Views. A adoção desta arquitetura permitiu desenvolver uma aplicação mais organizada, facilitando a manutenção, a reutilização de código e a implementação de novas funcionalidades.

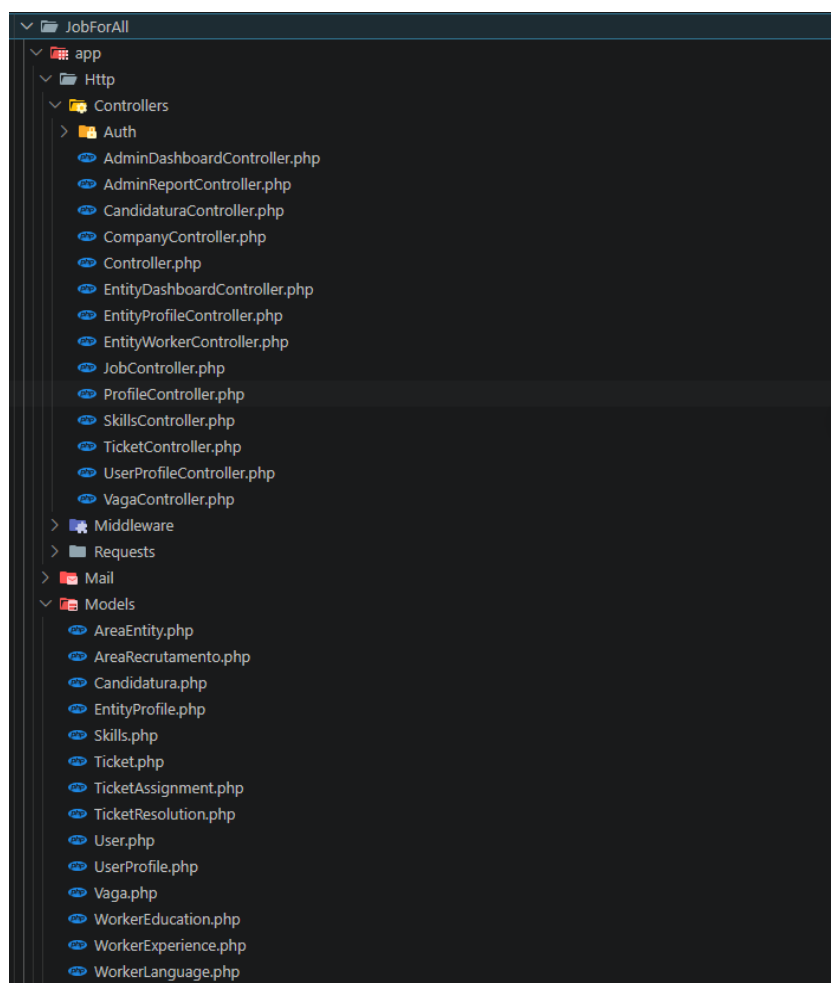


Figura 32 - Controllers e Models

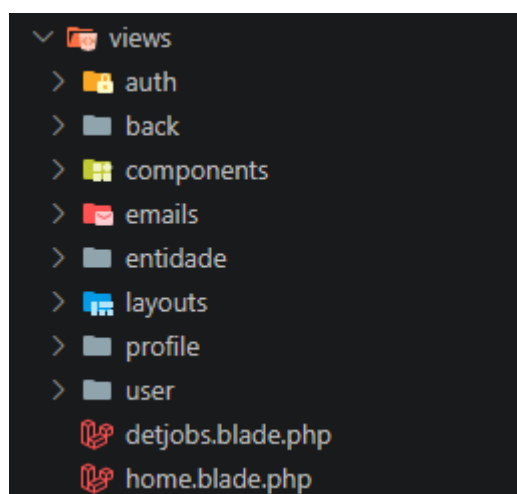


Figura 33 – Views

3.3.3. SEPARAÇÃO ENTRE FRONTOFFICE E BACKOFFICE

A aplicação foi organizada em duas áreas distintas: Frontoffice e Backoffice. O Frontoffice corresponde à área acessível aos utilizadores da plataforma, onde é possível consultar ofertas de emprego, efetuar registos, autenticar-se e utilizar as diferentes funcionalidades disponibilizadas pela aplicação.

O Backoffice destina-se à administração da plataforma, permitindo aos administradores gerir utilizadores, empresas, ofertas de emprego, categorias e restantes conteúdos necessários ao funcionamento do sistema. Esta separação permitiu definir diferentes níveis de acesso, aumentando a organização da aplicação e reforçando a segurança das funcionalidades administrativas.

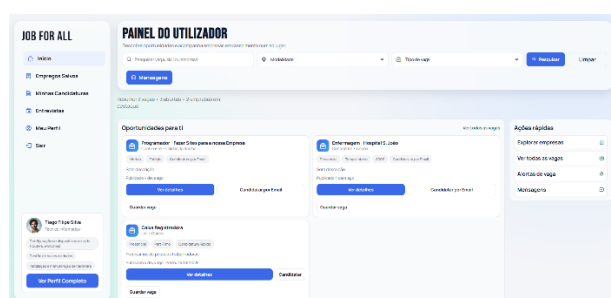


Figura 34 - Frontoffice

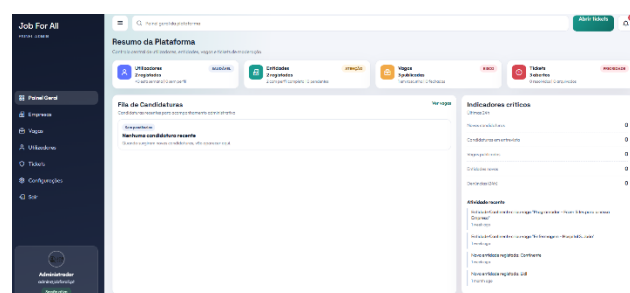


Figura 35 - Backoffice

3.4. DESENVOLVIMENTO DA BASE DE DADOS

A base de dados desempenha um papel fundamental no funcionamento da aplicação JobForAll, sendo responsável pelo armazenamento e gestão de toda a informação utilizada pelo sistema. Durante o desenvolvimento do projeto, recorreu-se ao sistema de migrações do Laravel para criar e gerir a estrutura da base de dados, garantindo uma organização consistente e facilitando futuras alterações.

3.4.1. CRIAÇÃO DAS MIGRAÇÕES

O Laravel disponibiliza um sistema de migrações que permite criar e modificar a estrutura da base de dados através de código, sem necessidade de recorrer diretamente ao gestor de bases de dados. No desenvolvimento do projeto foram criadas várias migrações, correspondentes às diferentes tabelas necessárias ao funcionamento da aplicação, incluindo utilizadores, empresas, ofertas de trabalho, candidaturas, categorias e restantes entidades do sistema.

Após a criação das migrações, estas foram executadas através do comando `php artisan migrate`, permitindo criar automaticamente todas as tabelas na base de dados. A utilização deste mecanismo facilitou a gestão da estrutura da base de dados e permitiu manter um histórico de todas as alterações efetuadas.

3.4.2. MODELOS (MODELS)

Os modelos (Models) foram desenvolvidos para representar as diferentes entidades da aplicação e estabelecer a comunicação entre o Laravel e a base de dados através do Eloquent ORM. Cada modelo corresponde a uma tabela da base de dados, permitindo efetuar operações como inserção, consulta, atualização e eliminação de registos de forma simples e organizada.

Durante o desenvolvimento do projeto foram criados modelos para as principais entidades da aplicação, nomeadamente utilizadores, empresas, ofertas de trabalho, candidaturas, categorias e restantes elementos necessários ao funcionamento da plataforma. Esta abordagem permitiu concentrar toda a lógica relacionada com os dados em classes específicas, promovendo uma melhor organização do código.

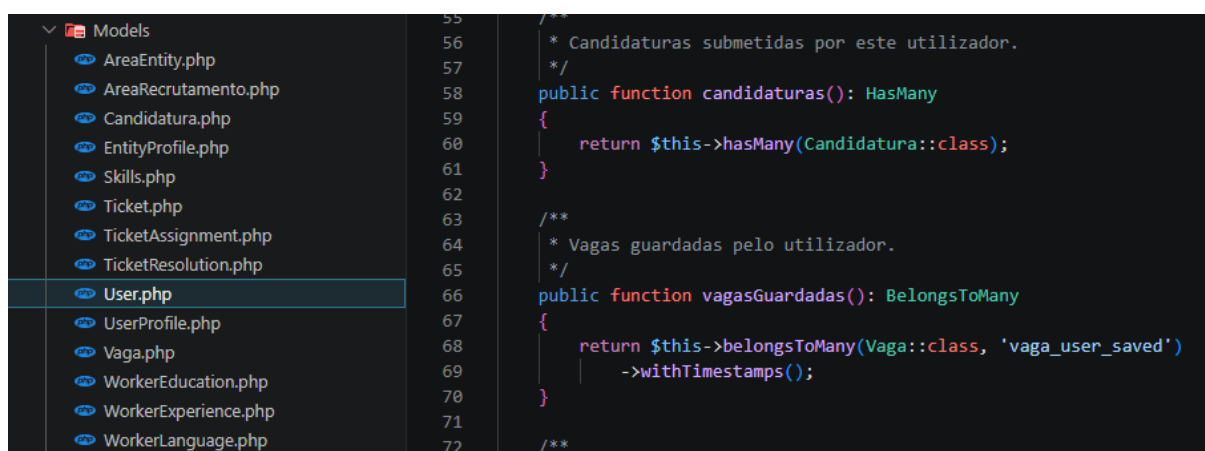


Figura 36 - Modelos utilizados na aplicação

3.4.3. RELACIONAMENTOS ENTRE TABELAS

A base de dados da aplicação foi desenvolvida com base em relacionamentos entre tabelas, permitindo estabelecer ligações entre as diferentes entidades do sistema e garantindo a integridade e consistência da informação armazenada. No Laravel, estes relacionamentos foram implementados através do Eloquent ORM, utilizando métodos específicos nos modelos, como `hasOne`, `hasMany`, `belongsTo` e `belongsToMany`. Esta abordagem permitiu representar as relações existentes entre as tabelas de forma simples e organizada, facilitando o acesso e a manipulação dos dados.

Durante o desenvolvimento do projeto foram definidos vários relacionamentos, dos quais se destacam:

- Um utilizador pode estar associado a uma empresa, sendo esta relação utilizada para identificar a empresa responsável pela publicação de ofertas de emprego.
- Uma empresa pode publicar várias ofertas de trabalho, enquanto cada oferta pertence apenas a uma empresa.
- Uma oferta de trabalho pode receber várias candidaturas, permitindo que diferentes utilizadores se candidatem à mesma vaga.
- Um utilizador pode submeter várias candidaturas, sendo cada candidatura associada a uma única oferta de trabalho.
- Os utilizadores podem ainda guardar ofertas de trabalho como favoritas, estabelecendo uma relação de muitos-para-muitos entre utilizadores e ofertas.

A utilização destes relacionamentos permitiu reduzir a redundância da informação, evitar a duplicação de dados e simplificar a realização de consultas à base de dados. Além disso, tornou o código mais organizado e facilitou o desenvolvimento das diferentes funcionalidades da aplicação, uma vez que o Laravel permite aceder aos dados relacionados através dos próprios modelos.

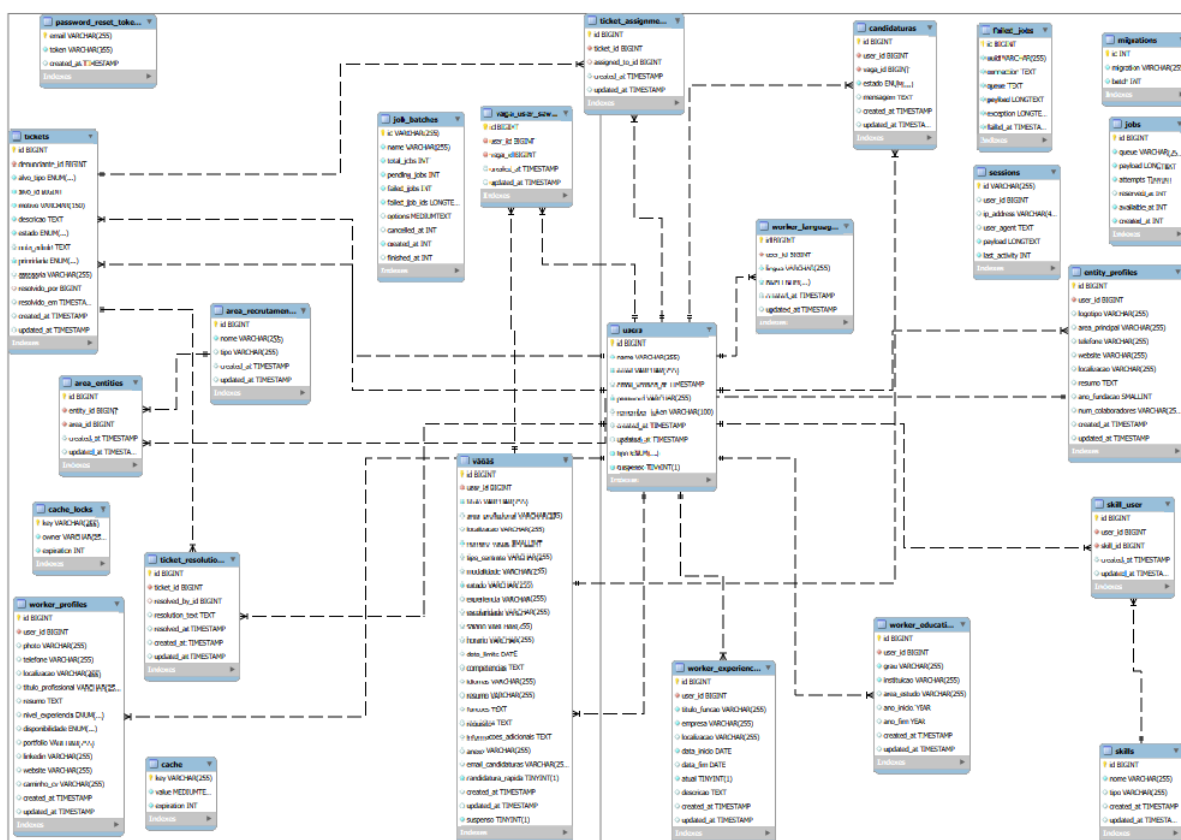


Figura 37 - Relacionamentos entre as tabelas da base de dados da aplicação JobForAll

3.5. SISTEMA DE AUTENTICAÇÃO

O sistema de autenticação foi desenvolvido para garantir que apenas utilizadores devidamente registados e autenticados possam aceder às funcionalidades privadas da aplicação. O Laravel disponibiliza mecanismos que facilitam a implementação deste sistema, assegurando a gestão de sessões, a proteção das rotas e o controlo de acessos de forma segura.

3.5.1. REGISTO DE UTILIZADORES (CANDIDATOS E EMPRESAS)

A aplicação permite o registo de dois tipos de utilizadores: candidatos e empresas. Durante o processo de registo, são solicitados os dados necessários à criação da conta, sendo posteriormente armazenados na base de dados. Cada tipo de utilizador possui permissões e funcionalidades específicas dentro da plataforma. Enquanto os candidatos podem pesquisar ofertas de emprego, submeter candidaturas e guardar vagas favoritas, as empresas têm acesso às funcionalidades relacionadas com a publicação e gestão das ofertas de trabalho.

Figura 38 - Formulário de registo de utilizadores

3.5.2. INÍCIO DE SESSÃO

Após a criação da conta, os utilizadores podem iniciar sessão através do formulário de autenticação da aplicação. Durante este processo, o sistema verifica se o endereço de correio eletrónico e a palavra-passe introduzidos correspondem aos dados armazenados na base de dados. Caso as credenciais estejam corretas, é criada uma sessão de utilizador, permitindo o acesso às funcionalidades autorizadas de acordo com o respetivo perfil.

O encerramento da sessão (Logout) também foi implementado, permitindo terminar a autenticação de forma segura e impedindo o acesso às áreas privadas após a saída da aplicação.

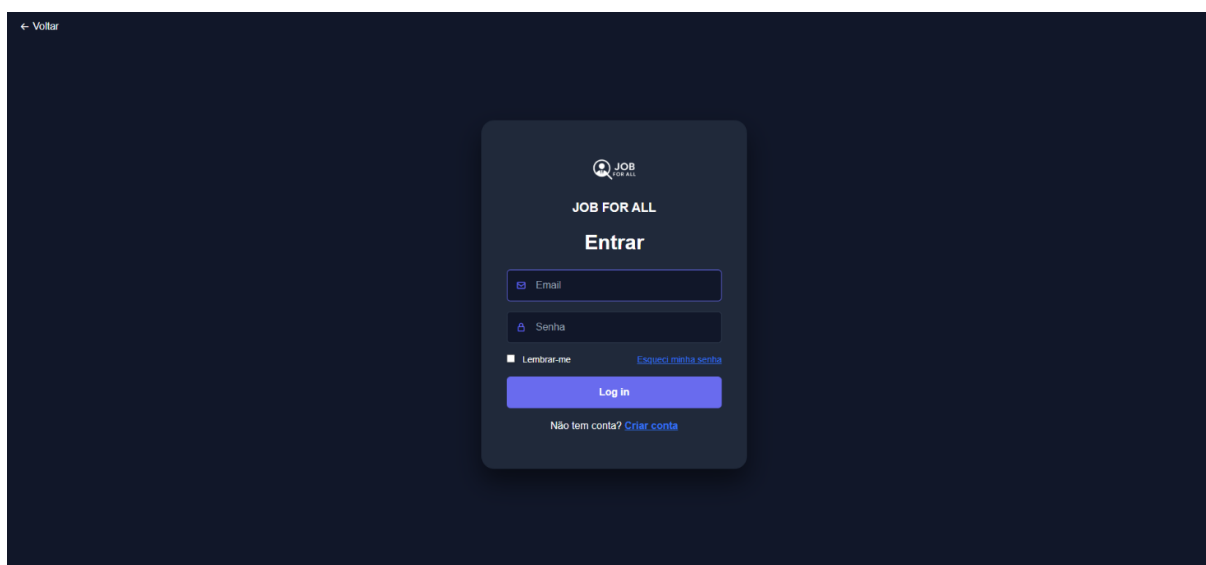


Figura 39 - Página de autenticação da aplicação

3.5.3. GESTÃO DE PERFIL DO UTILIZADOR

Após a autenticação, cada utilizador pode aceder ao seu perfil e atualizar as respetivas informações pessoais. Dependendo do tipo de conta, é possível editar dados como o nome, endereço de correio eletrónico, fotografia de perfil, contactos e outras informações relevantes. No caso das empresas, podem ainda ser alterados dados específicos, como a designação da empresa, a descrição e outros elementos associados ao perfil empresarial. Esta funcionalidade permite manter os dados sempre atualizados, contribuindo para uma melhor utilização da plataforma.

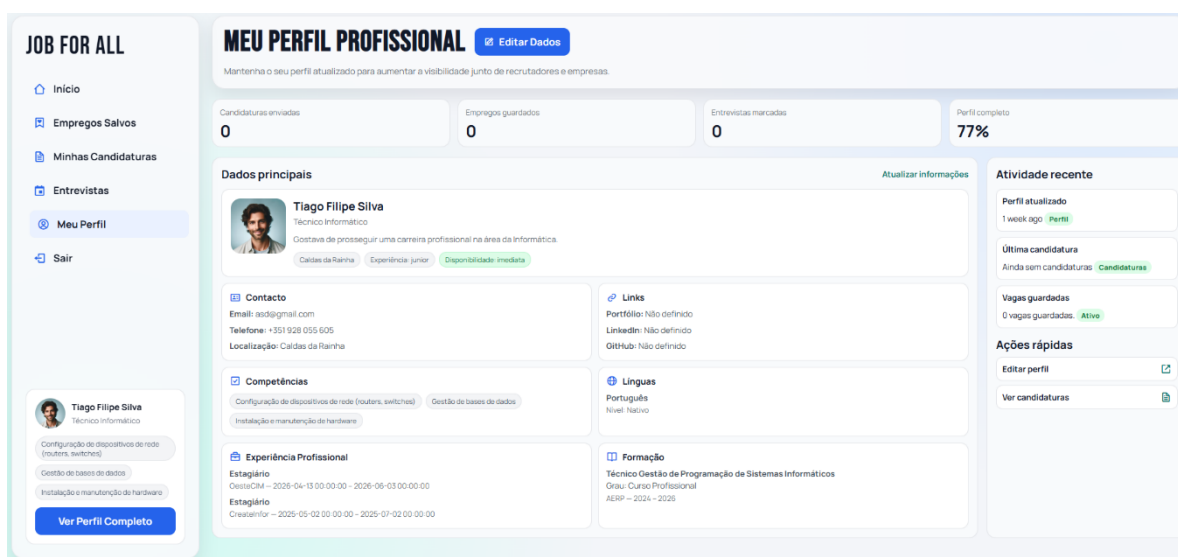


Figura 40 - Gestão do perfil do utilizador

3.5.4. PROTEÇÃO DAS ÁREAS PRIVADAS

As áreas privadas da aplicação foram protegidas através dos mecanismos de autenticação disponibilizados pelo Laravel. Foram utilizados *middlewares* para impedir o acesso de utilizadores não autenticados às páginas reservadas, como o painel de administração, a gestão de ofertas de trabalho, o perfil pessoal e outras funcionalidades exclusivas de utilizadores registados.

Além da autenticação, foram implementadas verificações de permissões para garantir que cada utilizador apenas pode aceder às funcionalidades correspondentes ao seu perfil, reforçando a segurança e a proteção dos dados da aplicação.

```
// Secção das entidades
Route::middleware(['auth', CheckRole::class.':entidade'])->group(function () {
    Route::get('/entidade', [EntityDashboardController::class, 'index'])->name('entidade.index');

    // Rotas para gerir trabalhadores (entidade)
    Route::get('/entidade/user', [EntityWorkerController::class, 'index'])->name('entidade.user');
    Route::get('/entidade/user/{user}', [EntityWorkerController::class, 'show'])->name('entidade.user.show');

    // Perfil da entidade e edição.
    Route::get('/entidade/profile', [EntityProfileController::class, 'show'])->name('entidade.profile');
    Route::get('/entidade/profile/edit', [EntityProfileController::class, 'edit'])->name('entidade.profile.edit');
    Route::post('/entidade/profile', [EntityProfileController::class, 'update'])->name('entidade.profile.update');

    // Listar, editar, eliminar e criar vagas.
    Route::get('/entidade/jobs', [VagaController::class, 'index'])->name('entidade.jobs');
    Route::get('/entidade/jobs/create', [VagaController::class, 'create'])->name('entidade.jobs.create');
    Route::post('/entidade/jobs', [VagaController::class, 'store'])->name('entidade.jobs.store');
    Route::get('/entidade/jobs/{vaga}/edit', [VagaController::class, 'edit'])->name('entidade.jobs.edit');
    Route::put('/entidade/jobs/{vaga}', [VagaController::class, 'update'])->name('entidade.jobs.update');
    Route::delete('/entidade/jobs/{vaga}', [VagaController::class, 'destroy'])->name('entidade.jobs.destroy');

    // Listar candidaturas recebidas pela entidade e permitir a gestão do estado.
    Route::get('/entidade/candidates', [CandidaturaController::class, 'entityIndex'])->name('entidade.candidates');
    Route::patch('/entidade/candidates/{candidatura}/status', [CandidaturaController::class, 'updateStatus'])->name('entidade.candidates.status');
});
```

Figura 41 - Proteção das áreas privadas através da autenticação da aplicação

3.6. SISTEMA DE PERMISSÕES

Para garantir um funcionamento seguro e organizado da plataforma JobForAll, foi implementado um sistema de permissões que controla o acesso às diferentes funcionalidades da aplicação. Cada utilizador possui um conjunto de permissões associado ao seu tipo de conta, permitindo que apenas tenha acesso às funcionalidades que lhe dizem respeito.

3.6.1. DEFINIÇÃO DOS TIPOS DE UTILIZADOR

A aplicação foi desenvolvida com três tipos principais de utilizador: Candidato, Entidade e Administrador. Cada um destes perfis possui diferentes permissões e responsabilidades dentro da plataforma. A distinção entre os diferentes tipos de utilizador é efetuada através de um campo específico na base de dados, permitindo ao sistema identificar automaticamente o perfil autenticado e disponibilizar as funcionalidades correspondentes.

Esta abordagem permitiu simplificar a gestão das permissões e assegurar que cada utilizador apenas acede às funcionalidades autorizadas.

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	BIGINT	☒	☒	☐	☐	☒	☐	☒	☐	
name	VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
email	VARCHAR(255)	☐	☒	☒	☐	☐	☐	☐	☐	
email_verified_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	NULL
password	VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
remember_token	VARCHAR(100)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
created_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	NULL
updated_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	NULL
tipo	ENUM('admin', 'entid...	☐	☒	☐	☐	☐	☐	☐	☐	'user'
suspenso	TINYINT(1)	☐	☒	☐	☐	☐	☐	☐	☐	'0'

Figura 42 - Definição dos tipos de utilizador no projeto

3.6.2. ÁREA DE CANDIDATO

A área destinada aos candidatos reúne todas as funcionalidades relacionadas com a procura de emprego. Após iniciar sessão, o candidato pode consultar ofertas de trabalho, efetuar pesquisas, guardar vagas como favoritas, submeter candidaturas, acompanhar o estado das candidaturas e atualizar as informações do seu perfil.

Todas estas funcionalidades encontram-se disponíveis apenas para utilizadores autenticados com o perfil de candidato.

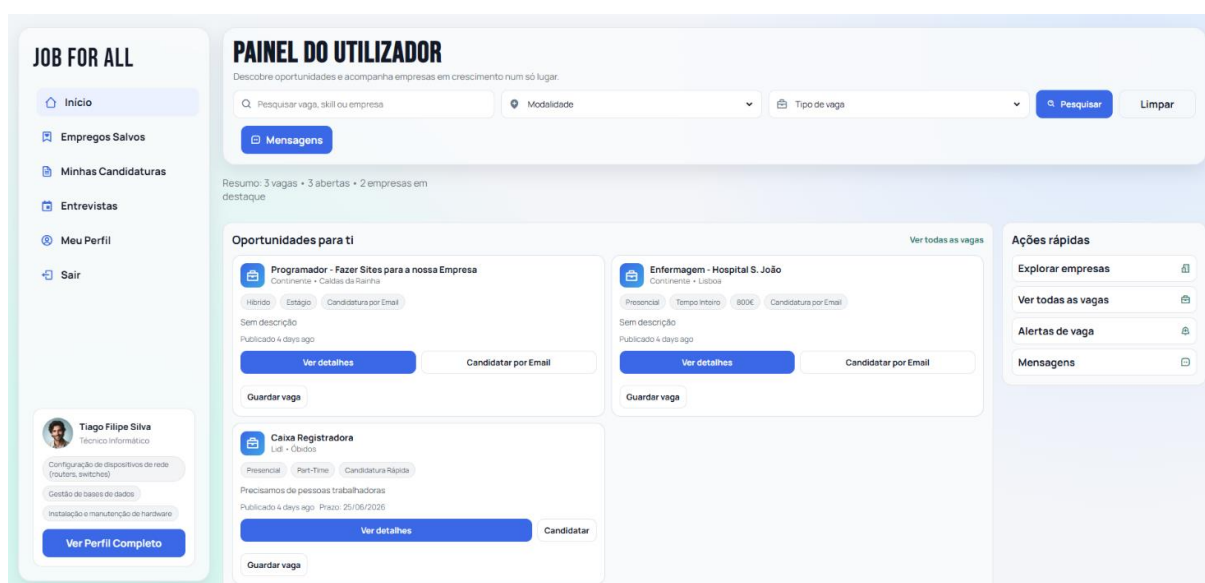


Figura 43 - Área reservada aos candidatos

3.6.3. ÁREA DE EMPRESA

A área de empresa foi desenvolvida para permitir às entidades empregadoras gerir os processos de recrutamento através da plataforma. Entre as principais funcionalidades encontram-se a criação, edição e remoção de ofertas de trabalho, a consulta das candidaturas recebidas e a gestão das informações da empresa. Esta área apenas se encontra acessível aos utilizadores autenticados com permissões de empresa.

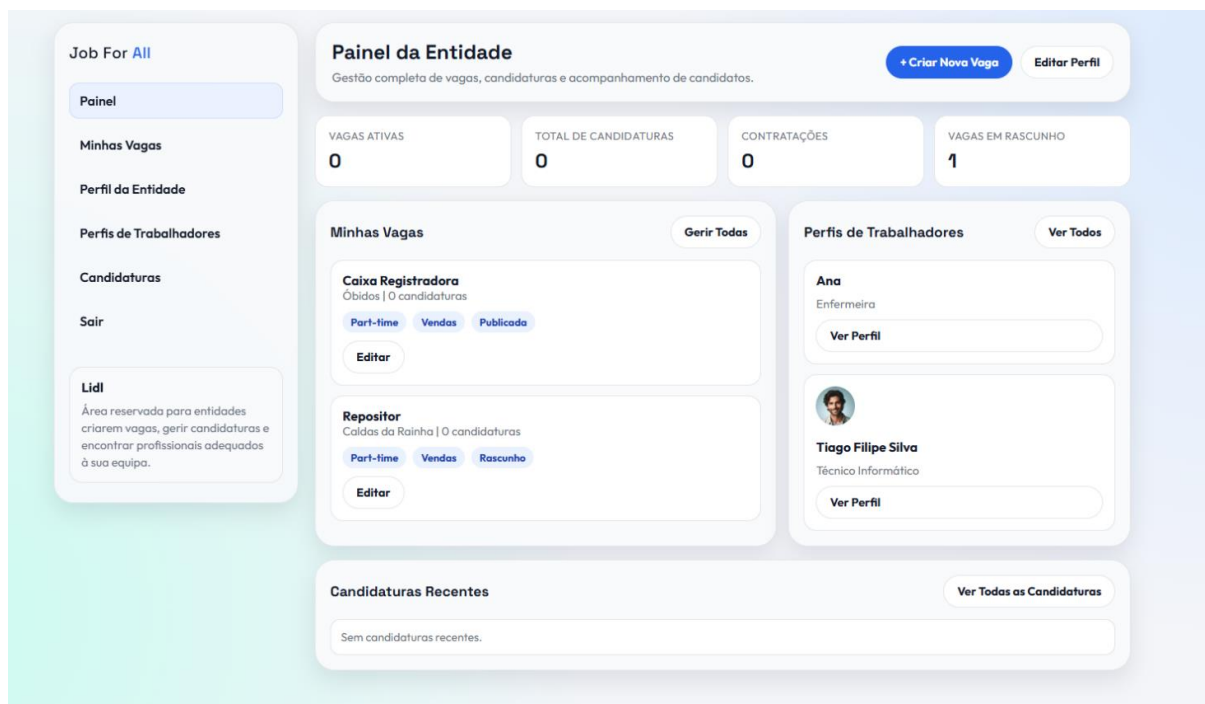


Figura 44 - Área reservada às empresas

3.6.4. ÁREA DE ADMINISTRADOR

A área de administração disponibiliza funcionalidades de gestão global da plataforma.

Através desta área é possível administrar utilizadores, empresas, categorias, ofertas de trabalho e restantes elementos necessários ao funcionamento do sistema. O administrador possui permissões alargadas, permitindo supervisionar e gerir toda a informação existente na aplicação.

O acesso a esta área encontra-se limitado exclusivamente aos utilizadores com perfil de administrador.

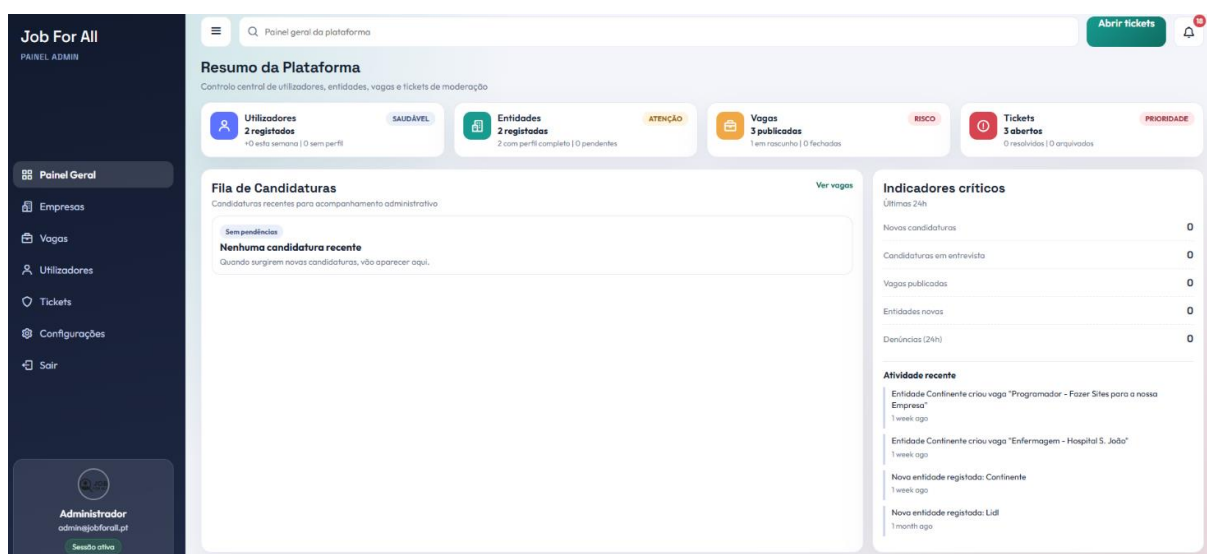


Figura 45 - Painel de administração do projeto

3.6.5. CONTROLO DE ACESSO AO BACKOFFICE

O acesso ao Backoffice é controlado através de verificações efetuadas pelo Laravel, garantindo que apenas os utilizadores autorizados podem aceder às funcionalidades administrativas. Sempre que um utilizador tenta aceder a uma área para a qual não possui permissões, o sistema impede o acesso e redireciona-o para uma página apropriada ou apresenta uma mensagem de erro. Desta forma, é assegurada a proteção das funcionalidades administrativas e da informação armazenada na plataforma.

Este mecanismo contribui para aumentar a segurança da aplicação e impedir acessos indevidos às áreas de gestão.

3.7. DESENVOLVIMENTO DO FRONTOFFICE (PÁGINA INICIAL DO UTILIZADOR E ENTIDADE)

O Frontoffice da aplicação JobForAll corresponde à interface principal disponibilizada aos utilizadores e entidades, permitindo a interação com as funcionalidades da plataforma.

Esta área foi desenvolvida com foco na usabilidade, organização da informação e facilidade de navegação, garantindo uma experiência intuitiva para os diferentes tipos de utilizador.

3.7.1. ÁREA DO UTILIZADOR (CANDIDATO)

A área do utilizador foi desenvolvida para permitir aos candidatos procurar e candidatar-se a ofertas de emprego de forma simples e eficiente. Na página inicial são apresentadas ofertas de emprego em destaque, bem como funcionalidades de pesquisa rápida, permitindo ao utilizador encontrar oportunidades de acordo com os seus interesses.

Os utilizadores podem consultar a listagem completa de ofertas, organizada por categorias e com sistema de paginação, facilitando a navegação entre os resultados. Foi implementado um sistema de pesquisa que permite filtrar ofertas por título, empresa ou localização, tornando a procura mais precisa.

Cada oferta possui uma página de detalhe, onde são apresentados todos os dados relevantes, incluindo descrição, requisitos, localização, tipo de contrato e possibilidade de candidatura. Relativamente às candidaturas, os utilizadores podem submeter o seu currículo diretamente através da plataforma, sendo posteriormente apresentada uma confirmação do envio.

Adicionalmente, os utilizadores podem guardar ofertas como favoritas, facilitando o acesso posterior às vagas de interesse.

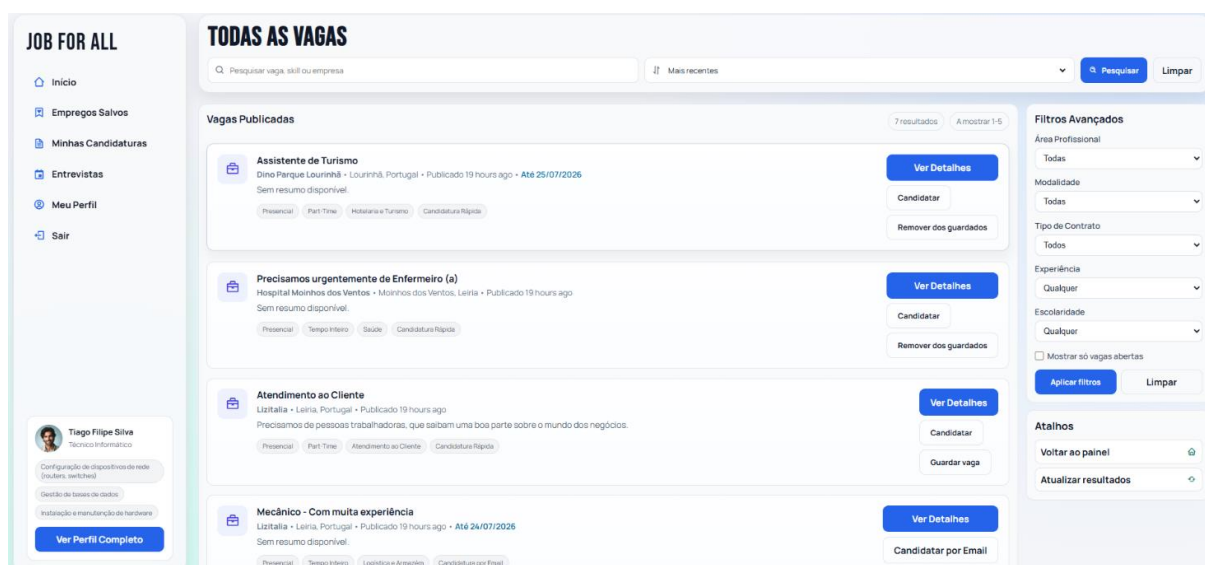


Figura 46 - Interface do utilizador no Frontoffice

3.7.2. ÁREA DA ENTIDADE (EMPRESA)

A área da entidade foi desenvolvida para permitir às empresas gerir as suas ofertas de emprego e acompanhar as candidaturas recebidas. As entidades podem criar, editar e remover ofertas de emprego, definindo informações como título, descrição, requisitos, localização e tipo de contrato.

Foi também implementada a funcionalidade de gestão de candidaturas, permitindo visualizar os candidatos associados a cada vaga. A interface foi organizada de forma a facilitar a gestão das ofertas, incluindo a possibilidade de visualizar o estado das mesmas, como ativas ou em rascunho.

Esta área permite às empresas gerir todo o processo de recrutamento de forma centralizada, tornando a plataforma uma ferramenta útil para a publicação e acompanhamento de oportunidades de emprego.

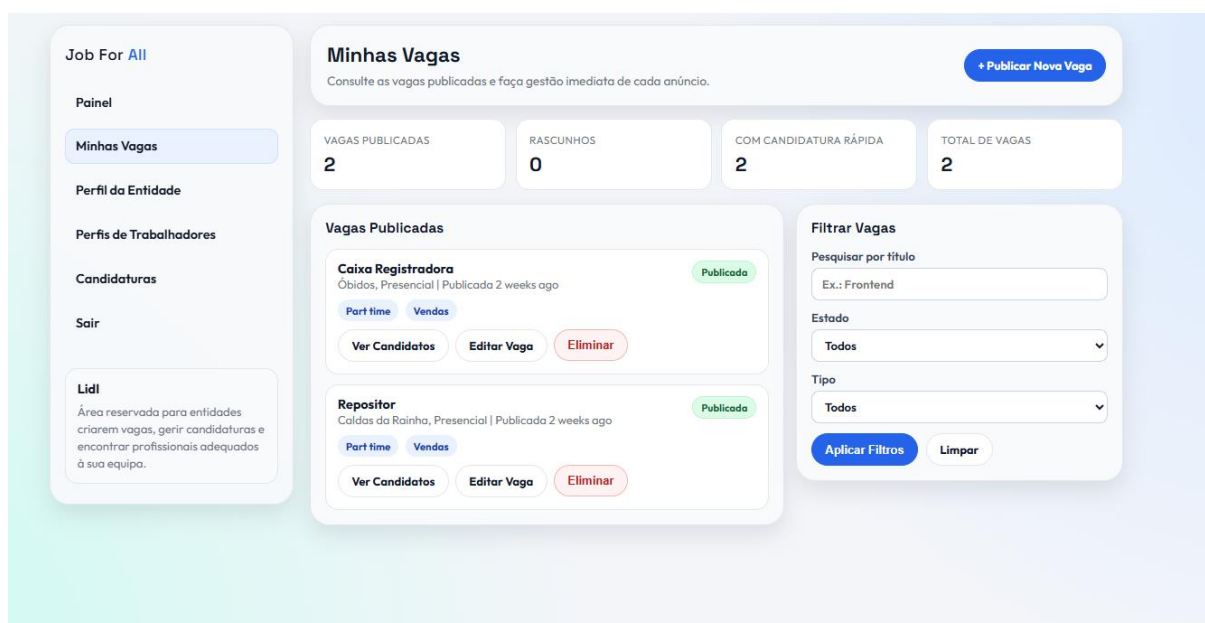


Figura 47 - Interface da entidade no Frontoffice.

3.8. DESENVOLVIMENTO DO BACKOFFICE (DASHBOARD)

O Backoffice da aplicação JobForAll foi desenvolvido com o objetivo de permitir a gestão completa da plataforma por parte dos administradores. Esta área disponibiliza um conjunto de funcionalidades que permitem controlar utilizadores, empresas, ofertas de emprego e outros elementos essenciais ao funcionamento do sistema.

3.8.1. DASHBOARD

O dashboard constitui a página inicial do Backoffice, apresentando uma visão geral do estado da aplicação JobForAll. Nesta área foram incluídos alguns elementos estáticos, responsáveis pela organização visual da interface, bem como indicadores dinâmicos que refletem dados reais da plataforma. Entre os principais indicadores apresentados encontram-se o número total de utilizadores, o número de entidades registadas, o total de ofertas de emprego publicadas e o número de denúncias ativas.

Para além destes indicadores, o dashboard destaca também informações relevantes relacionadas com as candidaturas. Sempre que existem candidaturas associadas a ofertas de emprego, estas são apresentadas de forma destacada, permitindo ao administrador visualizar rapidamente os candidatos às vagas disponíveis.

Caso não existam candidaturas registadas, esta secção não apresenta qualquer informação, mantendo a interface limpa e organizada. Esta abordagem permite fornecer ao administrador uma visão rápida e clara da atividade da plataforma, facilitando a tomada de decisões e a gestão eficiente do sistema.

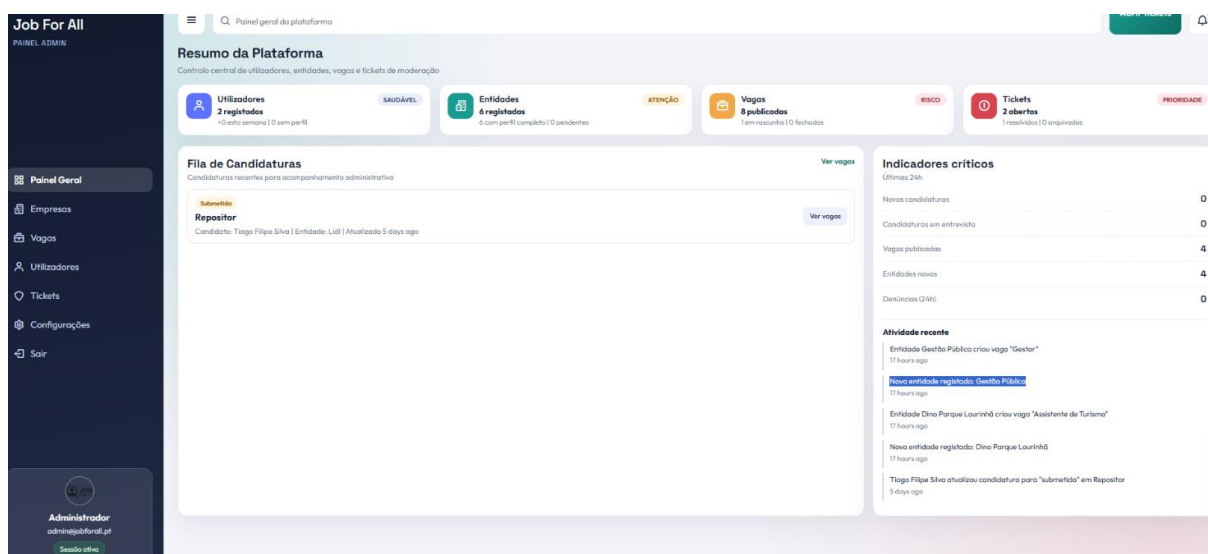


Figura 48 - Dashboard do Backoffice com indicadores e candidaturas

3.8.2. GESTÃO DE UTILIZADORES

A área de gestão de utilizadores permite ao administrador controlar todas as contas registadas na plataforma JobForAll. Nesta secção, é possível visualizar a lista completa de utilizadores, bem como aplicar filtros de pesquisa por nome ou endereço de correio eletrónico, facilitando a identificação de contas específicas.

À semelhança da gestão de empresas, o sistema permite identificar utilizadores que tenham sido alvo de denúncias, possibilitando a análise das situações reportadas. O administrador pode ainda consultar os dados associados a cada utilizador, bem como proceder à edição ou eliminação de contas, de acordo com as necessidades da plataforma.

Foram também implementadas funcionalidades de controlo, permitindo suspender ou reativar utilizadores, garantindo que apenas contas válidas e em conformidade com as regras permanecem ativas. Esta funcionalidade contribui para a gestão e moderação da comunidade da aplicação, assegurando um ambiente mais seguro e organizado.

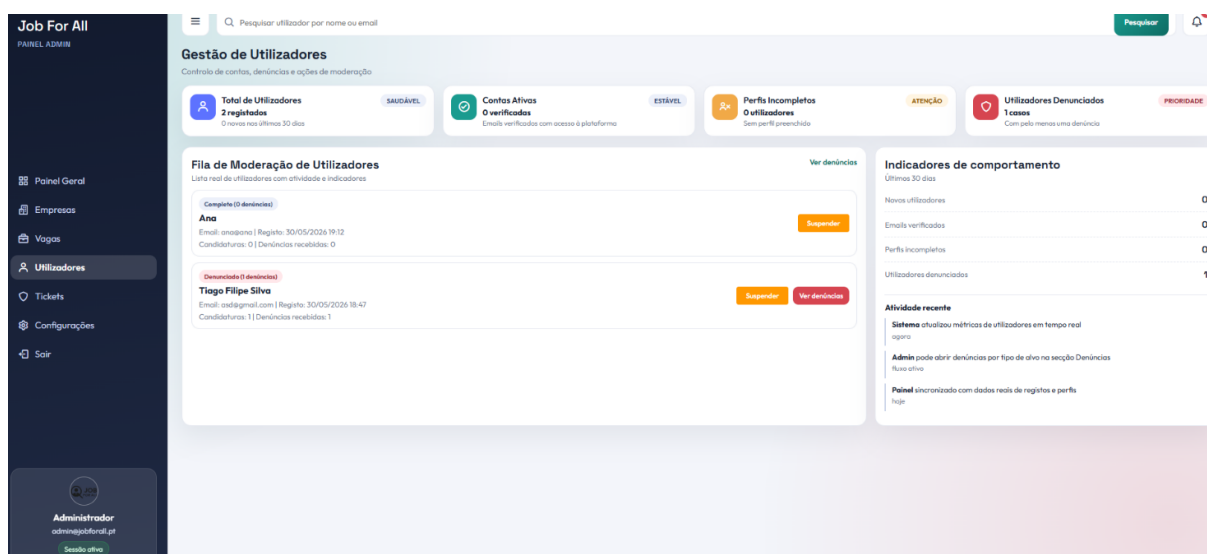


Figura 49 - Gestão de utilizadores com filtros e controlo de contas

3.8.3. GESTÃO DE EMPRESAS

A área de gestão de empresas permite ao administrador controlar todas as entidades registadas na plataforma JobForAll. Nesta secção, é possível visualizar a lista completa de empresas, bem como aplicar filtros de pesquisa por nome da empresa ou endereço de correio eletrónico, facilitando a localização de entidades específicas.

Para além disso, o sistema permite identificar empresas que tenham sido alvo de denúncias, disponibilizando uma listagem dedicada a entidades denunciadas. O administrador pode ainda consultar, de forma detalhada, as denúncias associadas a cada empresa, permitindo uma análise mais aprofundada das situações reportadas.

Com base nesta informação, foram implementadas funcionalidades de controlo, nomeadamente a possibilidade de suspender ou reativar contas de empresas. Esta abordagem permite agir sobre entidades que violem as regras da plataforma, garantindo um ambiente mais seguro e controlado.

Esta funcionalidade assume um papel importante na moderação do sistema, contribuindo para a qualidade e credibilidade da aplicação.

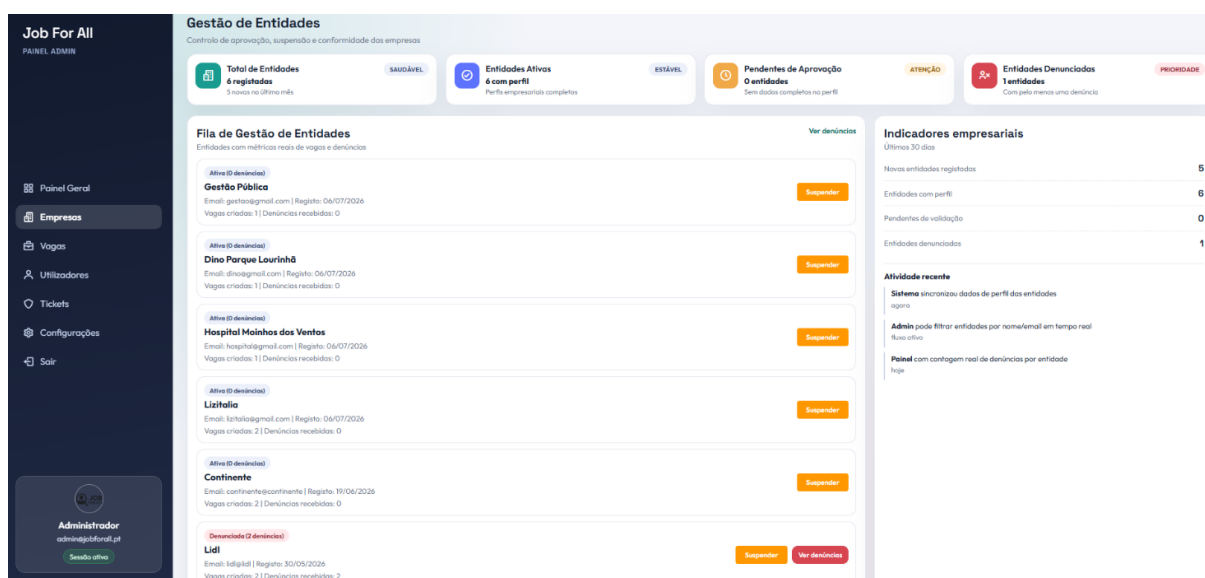


Figura 50 - Gestão de empresas com filtros e controlo de denúncias

3.8.4. GESTÃO DE OFERTAS DE EMPREGO

A área de gestão de ofertas de emprego permite ao administrador controlar todas as vagas publicadas na plataforma JobForAll. Nesta secção, é possível visualizar a lista completa de ofertas, bem como aplicar filtros de pesquisa por título da vaga, entidade responsável e localização, facilitando a organização e consulta da informação.

À semelhança da gestão de empresas, o sistema permite identificar ofertas que tenham sido alvo de denúncias, possibilitando ao administrador consultar detalhadamente as situações reportadas e tomar as medidas adequadas.

Adicionalmente, foi implementada a funcionalidade de visualização de ofertas em estado de rascunho, permitindo acompanhar vagas que ainda não foram publicadas. Esta funcionalidade facilita o controlo dos conteúdos antes de serem disponibilizados na plataforma.

O administrador dispõe ainda de mecanismos para suspender ou reativar ofertas de emprego, garantindo que apenas conteúdos apropriados permanecem ativos. Esta área assume um papel importante na moderação da plataforma, assegurando a qualidade e relevância das ofertas disponibilizadas aos utilizadores.

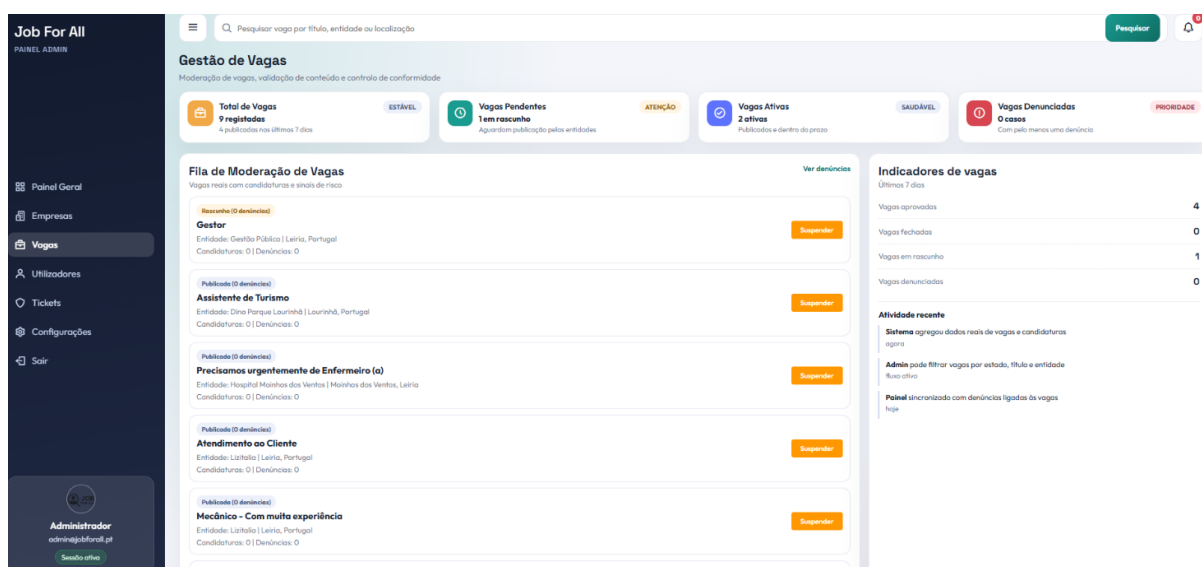


Figura 51 - Gestão de ofertas de emprego com filtros e controlo de estados

3.8.5. GESTÃO DE DENÚNCIAS

A área de gestão de denúncias permite ao administrador controlar e analisar todas as situações reportadas pelos utilizadores na plataforma JobForAll. Nesta secção, é possível visualizar a lista de denúncias, bem como aplicar filtros por estado e por tipo. Os estados disponíveis incluem pendente, em análise, resolvida e arquivada, permitindo acompanhar o ciclo de vida de cada denúncia. Relativamente ao tipo, as denúncias podem estar associadas a entidades, ofertas de emprego, candidaturas ou utilizadores.

Para além da listagem, o sistema apresenta indicadores dinâmicos, como o número de denúncias abertas e a distribuição por tipo, permitindo ao administrador ter uma visão geral da atividade de moderação. Cada denúncia pode ser consultada individualmente, disponibilizando informações detalhadas sobre a situação reportada. No entanto, para maior eficiência, foi também implementada a possibilidade de alterar diretamente o estado da denúncia a partir da listagem, sem necessidade de aceder à página de detalhe.

Durante esta ação, o administrador pode ainda adicionar uma nota opcional, permitindo registar informações relevantes sobre a decisão tomada. Esta funcionalidade contribui para uma gestão mais eficiente e rápida das denúncias, garantindo a moderação da plataforma e a manutenção de um ambiente seguro para os utilizadores.

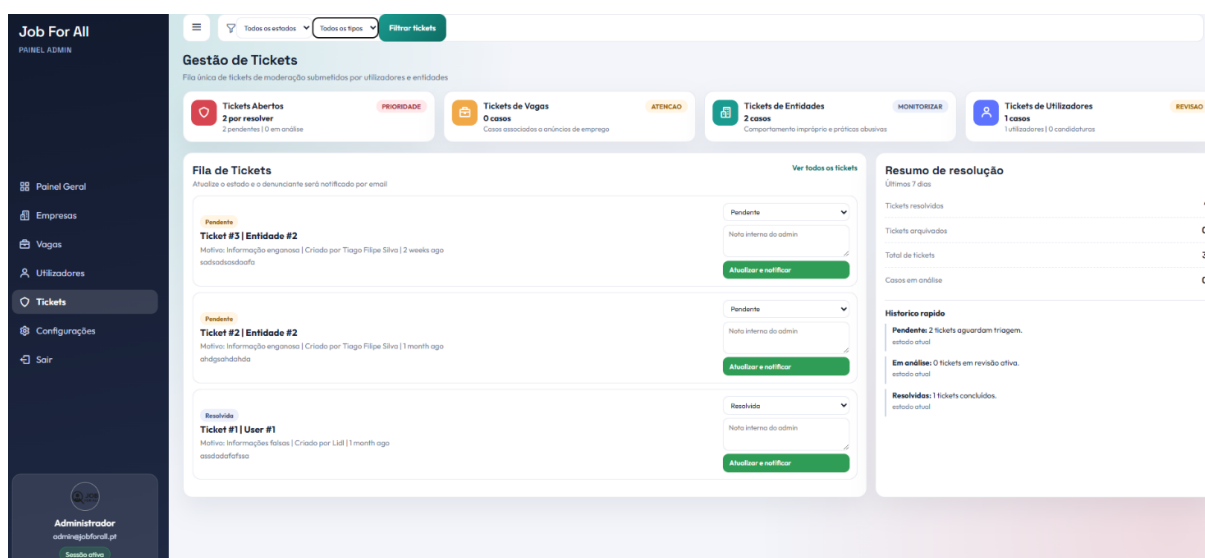


Figura 52 - Sistema de gestão de denúncias com filtros e controlo de estados.

3.9. DEFINIÇÃO DAS ROTAS

As rotas da aplicação foram definidas com o objetivo de encaminhar os pedidos dos utilizadores para os respetivos controladores, garantindo o correto funcionamento do sistema.

3.9.1. ORGANIZAÇÃO DO FICHEIRO WEB.PHP

O ficheiro web.php foi utilizado para definir as rotas principais da aplicação. Neste ficheiro foram organizadas rotas públicas, acessíveis a qualquer utilizador, e rotas protegidas, destinadas apenas a utilizadores autenticados.

As rotas foram agrupadas de forma lógica, separando funcionalidades relacionadas com candidatos, empresas e administração. Esta organização permitiu tornar o código mais estruturado e facilitar a manutenção do projeto. Além disso, foram utilizados middlewares para proteger determinadas rotas, garantindo que apenas utilizadores com permissões adequadas podem aceder a funcionalidades específicas.

```

3 use App\Http\Controllers\ProfileController;
4 use App\Http\Controllers\EntityWorkerController;
5 use App\Http\Middleware\CheckRole;
6 use App\Http\Controllers\UserProfileController;
7 use Illuminate\Support\Facades\Route;
8 use App\Http\Controllers\EntityProfileController;
9 use App\Http\Controllers\VagaController;
10 use App\Http\Controllers\JobController;
11 use App\Http\Controllers\CandidaturaController;
12 use App\Http\Controllers\CompanyController;
13 use App\Http\Controllers\EntityDashboardController;
14 use App\Http\Controllers\AdminDashboardController;
15 use App\Http\Controllers\AdminReportController;
16 use App\Http\Controllers\TicketController;
17
18
19 Route::get('/', function () {
20     return view('home', [
21         'vagasRecentes' => \App\Models\Vaga::with('user')
22             ->activeForUsers()
23             ->latest()
24             ->take(3)
25             ->get(),
26     ]);
27 })->name('home');
28
29 Route::get('/detjobs/{vaga}', function ($vagaId) {
30     $vaga = \App\Models\Vaga::findOrFail($vagaId);
31
32     // Impede o acesso a vagas que ainda não podem ser mostradas publicamente.
33     abort_unless($vaga->isVisibleForUsers(), 404);
34
35     $vaga->load(['user.entityProfile']);
36
37     return view('detjobs', compact('vaga'));
38 })->name('detjobs');
39
40 Route::middleware('auth')->get('/dashboard', function () {
41     $user = auth()->user();

```

Figura 53 - Organização das rotas no ficheiro web.php

3.10. DESENVOLVIMENTO DOS CONTROLADORES

Os controladores foram desenvolvidos com o objetivo de gerir a lógica da aplicação e tratar os pedidos recebidos através das rotas.

Cada controlador é responsável por uma área específica do sistema, como utilizadores, empresas, ofertas de emprego e candidaturas. Estes recebem os dados enviados pelos utilizadores, processam a informação e comunicam com os modelos para interagir com a base de dados. Foi implementado o padrão CRUD (Create, Read, Update, Delete), permitindo realizar operações completas sobre os dados da aplicação.

A utilização de controladores contribuiu para uma melhor organização do código, separando a lógica da aplicação da interface e facilitando a manutenção do sistema.

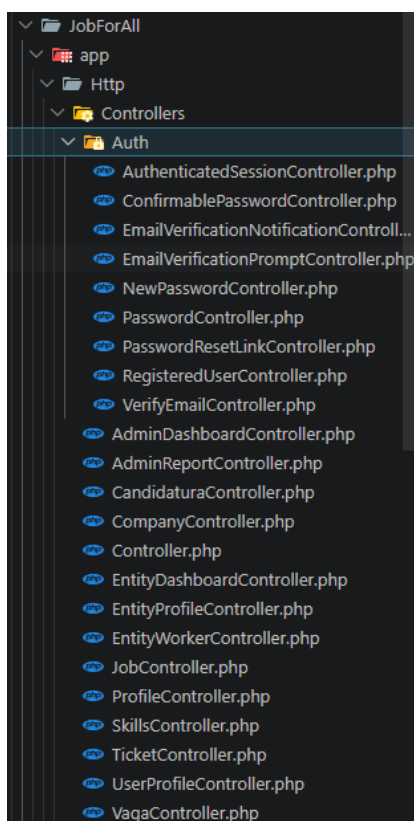


Figura 54 - Organização dos controladores da aplicação

3.11. DESENVOLVIMENTO DAS VIEWS

As views foram desenvolvidas com o objetivo de apresentar a interface da aplicação aos utilizadores, permitindo a interação com o sistema de forma clara e intuitiva.

Para a criação das views foi utilizado o motor de templates Blade, disponibilizado pelo Laravel, que permite construir páginas dinâmicas de forma organizada.

3.11.1. BLADE TEMPLATES

O Blade foi utilizado para estruturar as páginas da aplicação, permitindo reutilizar código e integrar facilmente dados provenientes dos controladores. Através deste sistema, foi possível criar templates dinâmicos, facilitando a apresentação de conteúdos como listas de ofertas, perfis de utilizadores e páginas de detalhe.

3.11.2. COMPONENTES REUTILIZÁVEIS

No desenvolvimento das views da aplicação, foram utilizados componentes reutilizáveis com o objetivo de garantir consistência visual e reduzir a duplicação de código. Foi criada uma estrutura

base na pasta layouts, onde se encontram os ficheiros principais da interface. Nestes ficheiros foi definido um layout geral da aplicação, incluindo elementos comuns como a barra de navegação (navbar), o rodapé (footer) e a estrutura principal das páginas.

Para permitir a inserção de conteúdo dinâmico, foi utilizada a diretiva `@yield('content')`, que define uma área onde o conteúdo específico de cada página é apresentado.

Nos restantes ficheiros de view, foi utilizada a diretiva `@extends('layouts.user')`, permitindo herdar o layout principal. O conteúdo específico de cada página é então definido através da diretiva `@section('content')`, sendo automaticamente inserido na área definida pelo `@yield`.

Esta abordagem permitiu manter uma estrutura organizada e consistente em toda a aplicação, facilitando a manutenção e a reutilização de código.

```

user.blade.php
<body>
  <div class="page">
    <aside class="sidebar">
      <div class="profile-box">
        <div class="profile-top">
          </div>
        <div class="skills">
          @foreach($layoutSkills->take(3) as $skill)
            <span class="chip">{{ $skill->nome }}</span>
          @endforeach
          @if($layoutSkills->count() > 3)
            <span class="chip chip-green">+{{ $layoutSkills->count() - 3 }}</span>
          @endif
        </div>
        <a class="btn btn-primary" href="{{ route('user.profile') }}">Ver perfil</a>
      </div>
    </aside>
    @yield('content')
  </div>
  <div class="report-modal" id="reportModal" aria-hidden="true">
    <div class="report-modal-backdrop" data-close-report-modal</div>
    <div class="report-modal-dialog" role="dialog" aria-modal="true" aria-labelledby="reportModalTitle">
      <div class="report-modal-head">
        <h3 id="reportModalTitle">Enviar denúncia</h3>
        <button type="button" class="btn btn-outline btn-icon-action" data-close-report-modal">
          <i class="bx bx-x"></i>
        </button>
      </div>
    </div>
  </div>

```

```

RELATORIO_PROJETO.txt
saved-jobs.blade.php
1 @extends('layouts.user')
2 @section('content')
3   @php
4     $mode = request('mode');
5   @endphp
6
7   <main class="settings-main">...
140 </main>
141 @endsection
142

```

Figura 55 - Utilização de layouts e templates Blade na aplicação

3.12. GESTÃO DE FICHEIROS (IMAGENS)

A aplicação permite o envio e gestão de ficheiros, nomeadamente imagens associadas aos utilizadores e entidades.

3.12.1. UPLOAD DE IMAGEM DOS CANDIDATOS E ENTIDADES

Foi implementada a funcionalidade de upload de imagens, permitindo aos candidatos e empresas adicionar imagens ao seu perfil. Durante o processo de upload, os ficheiros são validados e posteriormente armazenados no sistema.

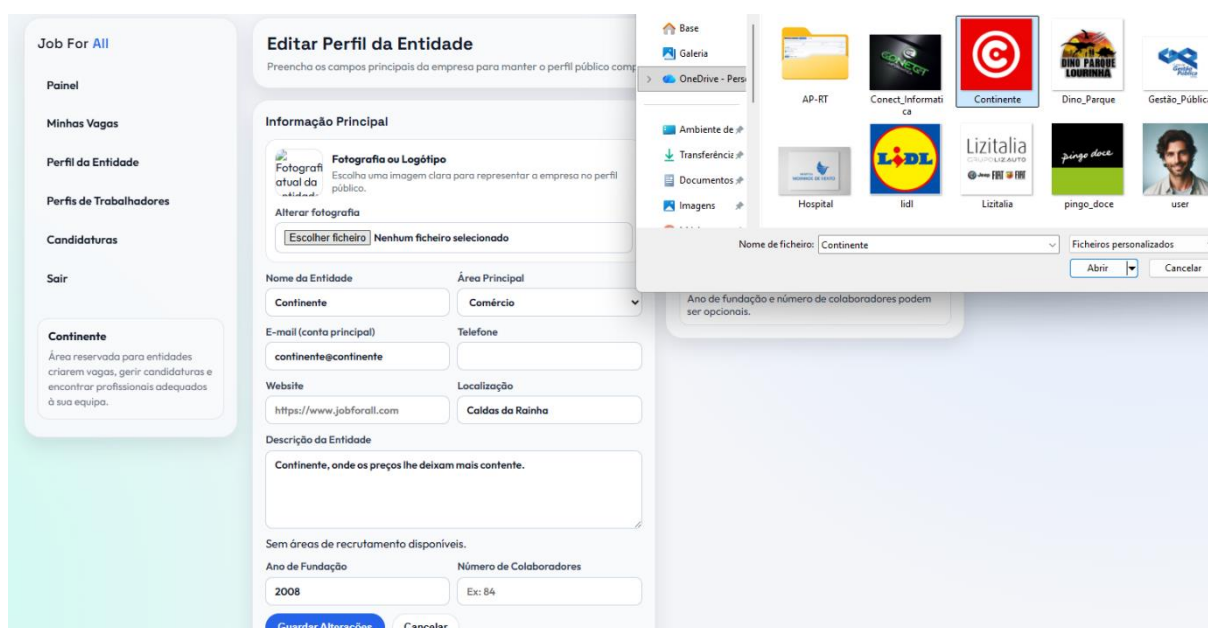


Figura 56 - Upload das imagens para foto de perfil

3.12.2. ARMAZENAMENTO NO LARAGON / STORAGE DO LARAVEL

As imagens enviadas são armazenadas na pasta storage do Laravel. Para permitir o acesso aos ficheiros através da aplicação, foi criada uma ligação simbólica entre as pastas storage e public, utilizando o comando `php artisan storage:link`.

Esta configuração permitiu disponibilizar as imagens no frontend, garantindo a sua correta visualização na aplicação.

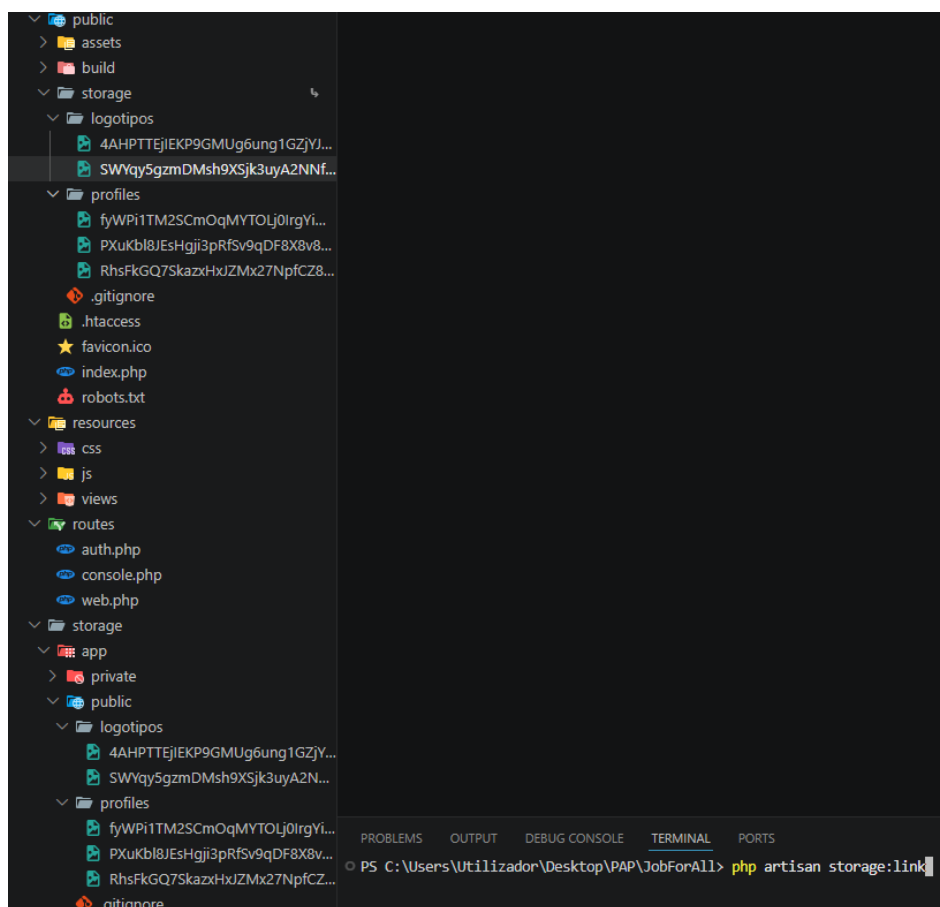


Figura 57 - Armazenamento e acesso às imagens na aplicação

3.13. VALIDAÇÃO DE DADOS

Durante o desenvolvimento da aplicação, foi implementado um sistema de validação de dados com o objetivo de garantir a integridade da informação introduzida pelos utilizadores. Foram definidas regras de validação nos formulários de registo, login, candidaturas e criação de ofertas, assegurando que os dados cumprem os requisitos estabelecidos.

Sempre que ocorre um erro, o sistema apresenta mensagens de feedback ao utilizador, indicando os campos que necessitam de correção. Adicionalmente, foi implementada validação nos uploads de ficheiros, garantindo que apenas formatos válidos são aceites.

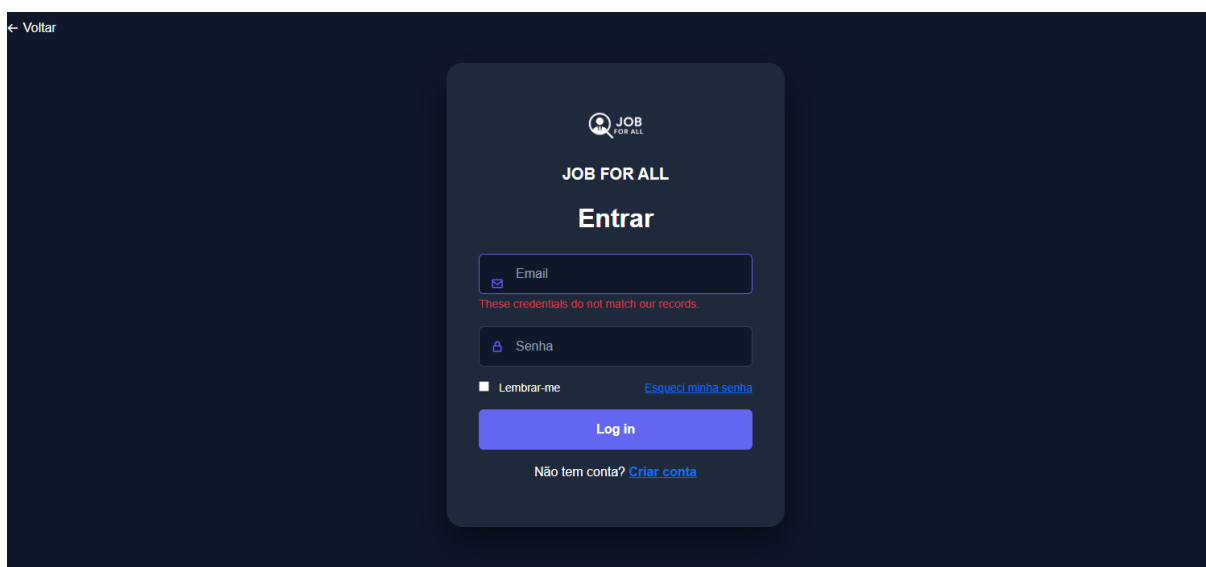


Figura 58 - Validação de dados

3.14. FUNCIONALIDADES COMPLEMENTARES

Para além das funcionalidades principais da aplicação, foram desenvolvidos mecanismos adicionais com o objetivo de melhorar a experiência do utilizador e tornar a plataforma mais completa e eficiente.

Estas funcionalidades permitem uma utilização mais prática da aplicação, facilitando a navegação, a organização da informação e a interação com as ofertas de emprego.

3.14.1. SISTEMA DE PESQUISA E FILTROS

Foi implementado um sistema de pesquisa que permite aos utilizadores encontrar ofertas de emprego de forma rápida e eficiente. A pesquisa pode ser realizada através de diferentes critérios, como o título da vaga, a empresa ou a localização. Esta funcionalidade permite reduzir o número de resultados apresentados, tornando a navegação mais simples e direcionada.

Para além da pesquisa, foram também aplicados filtros que possibilitam organizar os resultados de acordo com diferentes parâmetros, facilitando a identificação das ofertas mais relevantes para o utilizador. Este sistema contribui para uma melhor usabilidade da aplicação, permitindo encontrar oportunidades de forma mais eficaz.

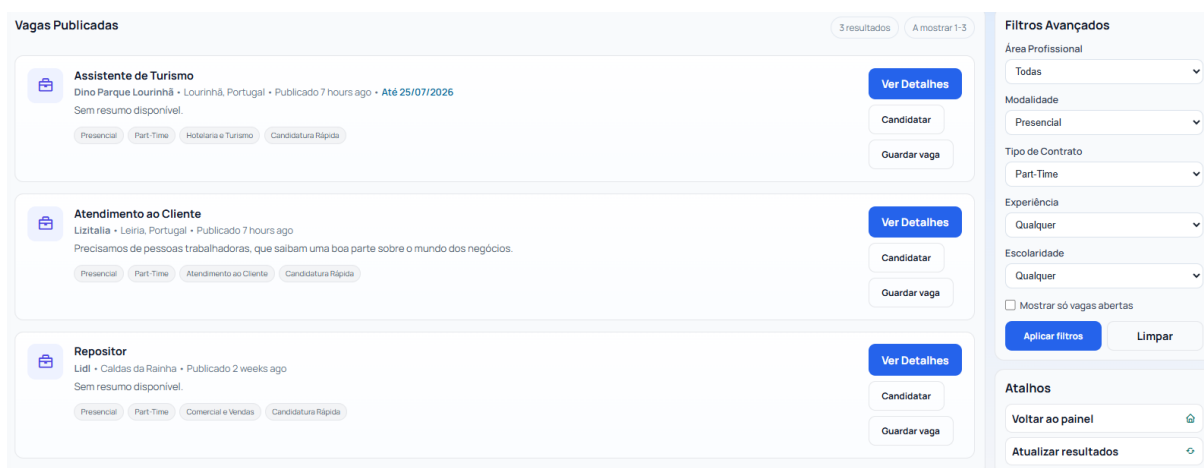


Figura 59 - Sistema de pesquisa e filtragem de ofertas de emprego

3.14.2. SISTEMAS DE FAVORITOS (GUARDAR VAGAS)

A aplicação permite aos utilizadores guardar ofertas de emprego como favoritas, possibilitando o acesso rápido às vagas de interesse. Esta funcionalidade foi desenvolvida através de uma relação entre utilizadores e ofertas de emprego, permitindo que cada utilizador possa guardar múltiplas vagas.

As ofertas guardadas podem ser consultadas posteriormente numa área dedicada, facilitando a organização das oportunidades e o acompanhamento das candidaturas. O sistema de favoritos contribui para uma melhor experiência do utilizador, permitindo uma gestão mais eficiente das ofertas selecionadas.

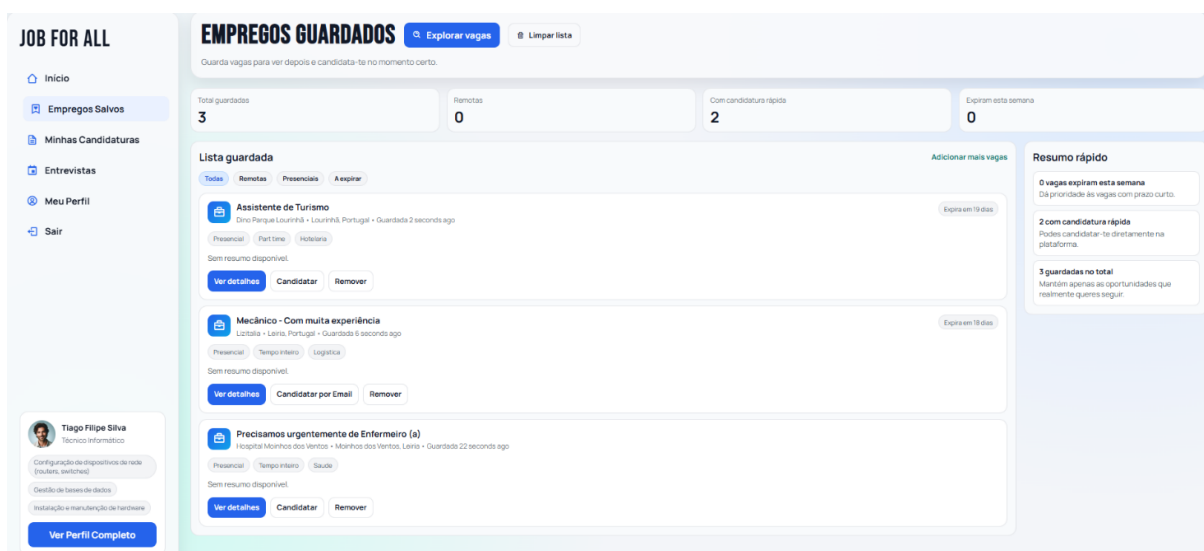


Figura 60 - Sistema de favoritos da aplicação

3.15. TESTES REALIZADOS

Durante o desenvolvimento do projeto, foram realizados diversos testes com o objetivo de garantir o correto funcionamento da aplicação.

Foram efetuados testes ao Frontoffice e Backoffice, verificando todas as funcionalidades disponíveis para os diferentes tipos de utilizador.

Testou-se também o sistema de autenticação, garantindo o correto registo, login e proteção das áreas privadas. Foram ainda realizados testes ao sistema de candidaturas, gestão de ofertas de emprego e gestão de utilizadores.

Sempre que foram identificados erros, procedeu-se à sua correção, contribuindo para a estabilidade e fiabilidade da aplicação.

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

Durante o desenvolvimento do projeto JobForAll, foram identificados diversos problemas técnicos, os quais foram analisados e resolvidos ao longo do processo de implementação. Estes desafios contribuíram para o aprofundamento dos conhecimentos e para a melhoria da qualidade final da aplicação.

Numa fase inicial, surgiu o erro "Class EntityWorkerController does not exist", causado pela ausência da importação do respetivo controlador no ficheiro de rotas (web.php). A resolução consistiu na adição da instrução `use App\Http\Controllers\EntityWorkerController;`, garantindo o correto funcionamento das rotas associadas.

Posteriormente, foram identificados problemas relacionados com as views, especialmente após alterações ao layout principal. Algumas páginas deixaram de funcionar devido à utilização de variáveis não definidas em todas as views, bem como à ausência de verificação de dados nulos. Para resolver esta situação, foram introduzidos valores de fallback e ajustada a lógica do layout, tornando a aplicação mais robusta.

Verificaram-se também erros associados à criação de perfis de novos utilizadores, causados por referências a dados inexistentes. A solução passou pela utilização de operadores de fallback (??) e pela adaptação da lógica de apresentação, garantindo um comportamento seguro na ausência de dados.

Ao nível da base de dados, foi identificado um problema na gravação de dados em tabelas relacionais, causado por inconsistências nos dados recebidos. Como correção, foi implementada a validação e normalização da informação antes do seu processamento.

Relativamente à apresentação de ofertas de emprego, verificou-se que vagas inativas ou suspensas continuavam a ser exibidas aos utilizadores. Para resolver esta situação, foram criados mecanismos de filtragem que garantem que apenas ofertas válidas e visíveis são apresentadas.



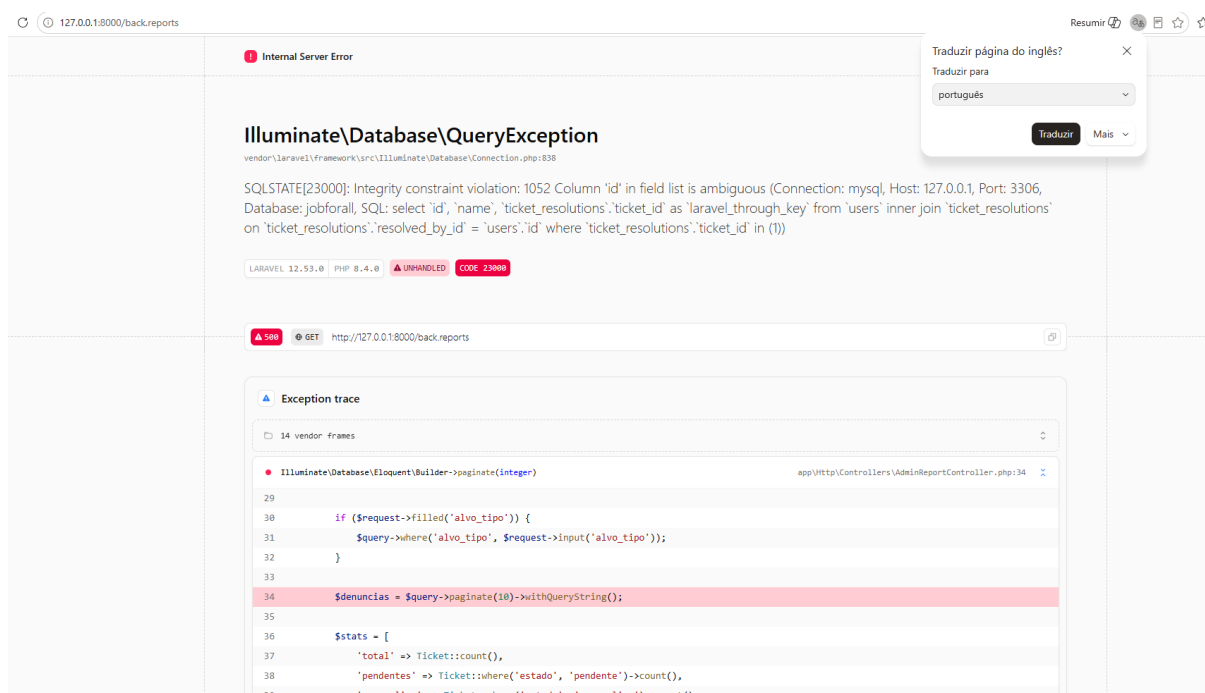


Figura 61 - Erro na coluna de gestão de denúncias

Conforme apresentado na Figura 61, foi identificado um erro durante o acesso à listagem e ao detalhe das denúncias na área administrativa da aplicação. A mensagem apresentada correspondia a uma violação de integridade SQL (SQLSTATE[23000]), indicando que a coluna id era ambígua na consulta executada.

Após análise, verificou-se que o problema estava relacionado com o carregamento de dados provenientes de múltiplas tabelas, onde existiam campos com o mesmo nome. Como a consulta não especificava a tabela de origem, o sistema não conseguia distinguir corretamente a coluna a utilizar.

Para resolver a situação, foi efetuada uma alteração na query, passando a identificar explicitamente os campos com a respetiva tabela de origem (por exemplo, users.id e users.name). Após a implementação desta correção, a ambiguidade foi eliminada e a funcionalidade voltou a operar corretamente, tanto na listagem como no detalhe das denúncias.

De forma geral, todos os problemas identificados foram resolvidos com sucesso, contribuindo para a estabilidade e fiabilidade da aplicação final.

5. CONCLUSÃO

Em suma, considera-se que o desenvolvimento deste projeto de PAP atingiu um nível de maturação bastante elevado, aproximando-se de uma versão final funcional e estável. Os objetivos definidos inicialmente foram cumpridos com sucesso, tendo sido possível desenvolver uma aplicação completa para gestão de ofertas de emprego, candidaturas e interação entre utilizadores e entidades.

Ao longo do desenvolvimento, foi possível aplicar e consolidar conhecimentos adquiridos nas áreas de programação web, bases de dados e desenvolvimento de aplicações, nomeadamente através da utilização do framework Laravel. A implementação seguiu uma estrutura organizada, baseada na arquitetura MVC, o que contribuiu para a clareza do código e para a facilidade de manutenção do sistema.

Para além das funcionalidades principais, foram também implementados sistemas complementares, como pesquisa e filtros, favoritos e gestão de denúncias, que enriquecem a experiência do utilizador e aproximam a aplicação de um contexto real. O desenvolvimento do Backoffice permitiu ainda criar ferramentas de controlo e moderação, essenciais para a gestão de uma plataforma deste tipo.

Durante o processo, foram identificados e resolvidos diversos problemas técnicos, o que contribuiu significativamente para o desenvolvimento de competências de análise, resolução de erros e pensamento crítico. Estas dificuldades permitiram uma melhor compreensão do funcionamento interno das tecnologias utilizadas.

Apesar do estado atual da aplicação ser funcional e estável, existem ainda algumas melhorias que poderão ser implementadas no futuro. Entre estas, destaca-se a substituição de elementos estáticos da interface por conteúdos dinâmicos mais completos, bem como a adição de novas funcionalidades.

Como perspetivas de evolução futura, prevê-se a implementação de um sistema de comunicação em tempo real (chat) entre utilizadores e entidades, permitindo uma interação mais direta dentro da plataforma. Adicionalmente, poderão ser desenvolvidos painéis de configuração mais avançados, permitindo uma personalização mais detalhada dos perfis.

De forma geral, este projeto constituiu uma experiência bastante enriquecedora, permitindo não só a aplicação de conhecimentos técnicos, mas também o desenvolvimento de competências ao

nível da organização, planeamento e autonomia. Considera-se, assim, que o projeto atingiu os objetivos propostos, resultando numa aplicação sólida e com potencial de evolução.

REFERÊNCIAS BIBLIOGRÁFICAS

As referências bibliográficas devem ser feitas de acordo com as Normas APA – 7.^a edição e ordenam-se por ordem alfabética dos apelidos dos autores.

Documentos na Web

Bootstrap. (s.d.). Bootstrap Documentation. <https://getbootstrap.com/docs/>

Indeed. (s.d.). <https://www.indeed.com/>

Laravel. (s.d.). Documentação oficial do Laravel. <https://laravel.com/docs>

LinkedIn. (s.d.). <https://www.linkedin.com/>

Mozilla Developer Network. (s.d.). Web Docs. <https://developer.mozilla.org/>

MySQL. (s.d.). MySQL Documentation. <https://dev.mysql.com/doc/>

Net-Empregos. (s.d.). <https://www.net-empregos.com/>

W3Schools. (s.d.). Web Development Tutorials. <https://www.w3schools.com/>

Vídeo do Youtube

Traversy Media, (16/11/2020). *Laravel Crash Course* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=MFh0Fd7BsjE>

FreeCodeCamp.Org, (07/05/2019). *Laravel PHP Framework Tutorial - Full Course for Beginners (2019)*

[Vídeo]. YouTube. <https://www.youtube.com/watch?v=ImtZ5yENzgE>

AsmrProg, (29/06/2024). *ASMR Programming - Job Platform UI Design - No Talking*

S [Vídeo]. YouTube. <https://www.youtube.com/watch?v=Cqofc2EeAMQ>

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: TIAGO FILIPE SILVA

N.º DE PROCESSO: 81568

CICLO DE ESTUDOS: SECUNDÁRIO

TELEMÓVEL: 928 055 605

E-MAIL: 2026ª081568@AERP.PT
